APPLICATION IMPLEMENTATION SUPPLEMENTAL GUIDE

Back to View

The materials presented in this technical guideline provide specific supplemental information into OUM for the purposes of application implementations. The Application Implementation Supplemental Guide contains the following components:

Application Implementation Overview

This component provides an overview for the application implementation views.

Application Implementation Process Guidelines

The Process Guidelines should be used in conjunction with the standard OUM process guidelines in order to accomplish application implementations.

Application Implementation Supplemental Activity Guidelines

The Supplemental Activity Guidelines should be used in conjunction with the standard OUM activity guidelines in order to accomplish application implementations.

Application Implementation Supplemental Task Guidelines

The Supplemental Task Guidelines should be used in conjunction with the standard OUM tasks in order to accomplish application implementations.

Oracle UPK Pro and Oracle Tutor Supplemental Guide

The materials presented in this tool guideline provide supplemental guidance for using Oracle® User Productivity Kit Professional (UPK Pro) and Oracle® Tutor in conjunction with OUM. Oracle Tutor provides users with tools to develop relevant process documentation and learning materials. Oracle UPK provides tools for annotated transaction simulations depicting proper field-by-field execution of key system processes. UPK Pro Knowledge Center provides an easy-to-search knowledge portal that provides stakeholders, project managers and employees with access to a central repository of all project assets. Used with available baseline content, the materials developed by these tools are process and role-based, easy to update, cost-effective and developed in conjunction with the implementation to accurately depict the future state model. Implementation documentation developed with UPK Pro and Tutor can be leveraged and used to create training content to enable end users to be productive

at go-live.

PDF View

You may also prefer to access this information through the **Application Implementation Supplemental Guide (PDF View)** located in the Supplemental Guides section of the Supplemental Guidance page.

Back to Top

PURPOSE

The purpose of this document is to provide an overview of the application implementation views for use for application implementation solutions, and to provide implementation teams with *supplemental guidance* for the successful planning and execution of application implementations.

Back to Top

AUDIENCE

This document is intended for practitioners who will be designing and implementing application implementation solutions based on the OUM (Oracle Unified Method) approach.

Back to Top

HOW TO USE THIS DOCUMENT

This document contains supplemental guidance for planning and executing application implementations and should be used in conjunction with and not instead of the OUM base method content.

Back to Top

LIMITATIONS TO THIS DOCUMENT

This document does not describe OUM in detail – the reader must review the documentation and training materials referenced below to gain a basic understanding of each.

Back to Top

RELATED DOCUMENTATION AND TRAINING

The following related documentation and training provide additional detailed guidance related to OUM implementations and are prerequisites to this document:

- OUM Method Pack – Desktop Installable or Hosted version
- OUM Training and Accreditation Program

Back to Top

CONTRIBUTORS

Bruce Dehner, Theresa Merenkov

Back to Top

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

APPLICATION IMPLEMENTATION OVERVIEW

Back to View

INTRODUCTION

An application implementation involves the definition and deployment of a business solution based on one or more commercial-off-the-shelf (COTS), packaged application products, or product suites, such as Oracle E-Business Suite or PeopleSoft Enterprise, etc. COTS application products are generally highly configurable, providing the flexibility needed to support a broad range of industry and regional requirements.

As a result, a primary focus of an application implementation is determining the most appropriate configuration for the specific business needs being addressed. In addition, there is typically some level of custom software development associated with an application implementation. Custom components may be developed, and integrated with the COTS package, to extend the functionality of the standard products, and/or to build integration between the new system and other COTS, or legacy systems. In some cases, significant custom software development is required to design, build and integrate major custom sub-components, such as a new application or Data Warehouse.

Back to Top

APPLICATION IMPLEMENTATION AND OUM

OUM provides two application implementation views to support commercial-off-the-shelf (COTS), packaged application implementations.

- The OUM Requirements-Driven Application Implementation View is a requirements-driven approach to implementing packaged applications in that it is based on identifying requirements early in a project through interviews, process modeling workshops, etc., and crafting a business solution based on those requirements during the course of the project.
- The OUM Solution-Driven Application Implementation View is a solution-driven approach to implementing packaged applications. A “solution-driven” approach is one based on the use of a pre-defined business solution (e.g., Oracle Business Accelerators) as the proposed business solution and tailoring that solution to the customer's specific requirements during the project.

Both of the OUM application implementation views are based on the underlying OUM Full Method Work Breakdown Structure (WBS), though each employs a different subset of the overall WBS that has been tailored to the needs of COTS application implementation projects, as well as the unique aspects of each approach. Both views also adhere to the five main principles of OUM.

The Solution-Driven Application Implementation view differs from the Requirements-Driven Application Implementation view primarily due to the use of a pre-defined solution as the starting point for the implementation, and the implicit goal inherent in the approach of maximizing the use of standard functionality and minimizing the need for custom extensions. Accordingly, the WBS for the Solution-Driven Application Implementation View contains a reduced number of activities and tasks related to custom

development – particularly in the early phases. It also assumes that the depth to which the remaining tasks are executed is reduced, when compared to the requirements-driven approach, because the foundational elements of the proposed solution have already been defined. Where many work products may need to be developed from scratch in a requirements-driven approach, the review of pre-existing assets associated with the pre-defined solution offering may be all that's necessary in a solution-driven approach.

Back to Top

KEY CONCEPTS

There are several key concepts that are important to understand in order to fully appreciate the OUM application implementation approaches. These concepts include:

- Business Process Models and Use Cases for Requirements Gathering
- Business Requirements Mapping
- Configuration Prototyping

Business Process Models and Use Cases for Requirements Gathering

The OUM Application Implementation approach employs techniques such as interviews and workshops to gain a comprehensive understanding of business requirements, which are documented using business process models and use cases. Use cases complement business process models in a number of important ways. See the OUM Method Overview to understand more about the relationship between use case modeling and business process modeling.

Business Requirements Mapping

In a commercial off-the-shelf (COTS) application implementation, business requirements are mapped to the COTS application system's capabilities in the task, Map Business Requirements (MC.030) in the Mapping and Configuration process. The application setup parameters required to support the detailed requirements reflected in the Future Process Model (RD.011.2) are determined during Gather Setup Information (MC.040) and documented in the Application Setup Documents (MC.050). Reporting requirements are also mapped to the standard reporting capabilities of the system during the task, Conduct Reporting Fit Analysis (MC.090).

Gaps in functionality are identified during the course of gathering business requirements and are initially assessed in Map Business Requirements (MC.030) and then more thoroughly evaluated and resolved in the tasks, Define and Estimate Custom Extensions (MC.100) and Define Gap Resolutions (MC.110), respectively.

Configuration Prototyping

The Setup Information (MC.040), Application Setups (MC.050) and the report mapping (MC.090) are validated and refined during the Configuration Prototyping Workshop (MC.080) to make sure the packaged application functionality supports the implementing organization's requirements. During this workshop, the project team demonstrates the configuration that was initially determined based on the results of the Specify Software Configuration activity, in order to validate that the configured business solution supports the business requirements as reflected in the future state process models and use cases.

If a pre-defined configuration is used as a starting point for the implementation, mapping may be performed primarily during validation of the configuration as described in the paragraph above.

Back to Top

KEY BEST PRACTICES

Overall Application Implementation Practices

- **Clear Understanding of Business Needs** - Before starting a project, do not assume that the organization has a clear understanding of its needs. Even if there is a

clear understanding of the perceived business problems to be addressed, there may be additional undocumented issues. Your team may need to work with the organization in a series of workshops to help determine their real needs and the root causes of their problems.

- **Upper Management Support** - Upper management has several key responsibilities that are connected to project success which include:
  - sponsoring the project and the business process change program resulting from the project
  - clearly defining project scope
  - resolving major issues in a timely manner
  - allocating resources
  - instilling a positive attitude throughout the organization
  - establishing project priority
  - managing the organizational changes associated with the project
  - verifying that project management has critical information about major planned or potential events that could impact the project
- **Strong Project Management** - A strong project manager identifies problems, determines proposed solutions, and drives the solution process. An inexperienced or ineffective manager may be unable to identify problem areas that could escalate and negatively impact the project.
- **Team Strength** - Capitalizing on individual and team strengths, while compensating for weaknesses, are critical factors affecting the project's success. Potential team factors include:
  - a strong team with weak project management may succeed, but will run a greater risk than a weak team with strong project management, who will have more potential to become a strong, effective project team
  - a group of strong individuals does not guarantee a strong team; teamwork is essential
- **Organizational Readiness** - Application implementations cause change, and effective change management may mean the difference between project success and failure. Project managers should be aware of organizational factors impacted by the project. For example:
  - Do various levels in the organization view the project as a desirable solution to current problems?
  - Is the organization's staff prepared to accept the changes to processes and organization that may result from the project?
  - Is there an organizational change management function in place to work with the project team in changing business processes and determining the necessary communication and reskilling strategies?
  - Is the information systems department able to absorb new technology and support the project?
  - What is the level of computer literacy? How many people will be using an automated system for the first time?
  - How stable is the organization? Change may be difficult for the user community if it is also being driven by other initiatives.
  - Are all levels of management ready? Authorizing a system implementation is different from managing the associated organizational changes.
  - Did the user community participate in the system selection? Greater involvement may mean less resistance to subsequent changes.
  - If the applications will be deployed at various sites, will business functions be performed differently at each location?
  - What is the organization's strategy regarding centralized versus decentralized planning and control?
  - What is the history of system implementation and has it created a positive mind set regarding future projects?

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

APPLICATION IMPLEMENTATION PROCESS GUIDELINES

Back to View

This document contains OUM supplemental process guidance for Application Implementation projects.

Keep in mind that not ALL OUM tasks are included in the Application Implementation Views. Refer to the Requirements-Driven Application Implementation View and the Solution-Driven Application Implementation View for a complete list of included tasks for each view.

BUSINESS REQUIREMENTS (RD) PROCESS GUIDANCE

Approach

In a commercial off-the-shelf (COTS) application implementation, business requirements are used as input to the high-level Future Process Model (RD.011.1) and subsequently the more detailed Future Process Model (RD.011.2).

During Business Requirements, you:

- Examine current processes and practices to understand and document the main business factors that currently benefit the business, if in scope for your project.
- Carefully document audit and control requirements to satisfy financial and quality policies.
- Identify the business operating requirements that the technical architecture will need to support.

The Use Case Specification (RA.024), which is prepared in the Requirements Analysis (RA) process, defines the necessary steps for executing a business function within a business process; some of the steps may be application-specific, and some may be manual. To confirm that business needs are met, you will also later map these requirements against application functions and features in Analysis.

Back to Top

REQUIREMENTS ANALYSIS (RA) PROCESS GUIDANCE

Approach

In a commercial off-the-shelf (COTS) application implementation, business requirements reflected in the Future Process Model (RD.011.2) are refined and structured via a Use Case Model in order to establish a more precise understanding of the requirements. At a minimum, Use Case Models and Use Case Specifications should be prepared for those requirements that can only be satisfied by custom-built components, which extend the functionality of the COTS system and/or integrate the COTS

system with other COTS or legacy systems. Preparation of Use Case Models and Use Case Details for all other business requirements is considered best practice, as it will contribute to a better understanding of requirements and can be leveraged to prepare a number of other required work products downstream, such as test scenarios and business procedures documentation.

Back to Top

ANALYSIS (AN) PROCESS GUIDANCE

Approach

The following tasks are generally performed only when there is a need to gain further clarification of the business requirements represented in the use cases:

- Analyze Data (AN.050)
- Analyze Behavior (AN.060)
- Analyze User Interface (AN.090)
- Prepare Analysis Specification (AN.100)
- Review Analysis Model (AN.110)

This typically applies to those requirements that can only be satisfied by custom-built components, which extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

Back to Top

DESIGN (DS) PROCESS GUIDANCE

Approach

In a commercial off-the-shelf (COTS) application implementation, a number of tasks in the Design process are performed only for those requirements that can only be satisfied by custom-built components, which extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

Back to Top

IMPLEMENTATION (IM) PROCESS GUIDANCE

Approach

In a commercial off-the-shelf (COTS) application implementation, tasks in the Implementation process are performed only for those requirements that can only be satisfied by custom-built components, which extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

APPLICATION IMPLEMENTATION SUPPLEMENTAL ACTIVITY GUIDELINES

Back to View

The Activity Guidelines are meant to assist the consultant in performing certain key tasks in an OUM-based Application Implementation projects. OUM defines an activity as "A set of tasks related either by topic, dependencies, data, common skills or work products."

Use the Application Implementation Supplemental Activity guidance provided on the left side of this table, in conjunction with the standard OUM Activity overview on the right side of the table, in order to understand how Application Implementation impacts specific OUM activities.

Activity ID	Application Implementation Supplemental Activity Guidelines	OUM Activity Overview (Links in this column refer to the OUM Activity Guidelines files)
Inception		
A.ACT.GRBI	Gather Business Requirements - Inception	Gather Business Requirements - Inception
A.ACT.CCPI	Create Conceptual Prototype - Inception	Create Conceptual Prototype - Inception
Elaboration		
B.ACT.DUCM	Develop Use Case Model	Develop Use Case Model
B.ACT.DUCD	Develop Use Case Details	Develop Use Case Details
B.ACT.AE	Analyze - Elaboration	Analyze - Elaboration
B.ACT.DE	Design - Elaboration	Design - Elaboration
B.ACT.DCCTSE	Develop Custom Component Test Scripts - Elaboration	Develop Custom Component Test Scripts - Elaboration
B.ACT.DVCSP	Develop and Validate Custom Software Prototypes	Develop and Validate Custom Software Prototypes
B.ACT.PCCTE	Perform Custom Component Testing - Elaboration	Perform Custom Component Testing - Elaboration
B.ACT.PACDE	Prepare to Acquire and Convert Data - Elaboration	Prepare to Acquire and Convert Data - Elaboration
Construction		
C.ACT.FR	Finalize Requirements	Finalize Requirements
C.ACT.AC	Analyze - Construction	Analyze - Construction
C.ACT.DC	Design - Construction	Design - Construction

C.ACT.PEC	Prepare Environments - Construction	Prepare Environments - Construction
C.ACT.DCCTSC	Develop Custom Component Test Scripts - Construction	Develop Custom Component Test Scripts - Construction
C.ACT.ICC	Implement Custom Components	Implement Custom Components
C.ACT.PCCTC	Perform Custom Component Testing - Construction	Perform Custom Component Testing - Construction
Transition		
Production		
E.ACT.RPP	Resolve Production Problems	Resolve Production Problems

Back to Top

INCEPTION

A.ACT.GBRI Gather Business Requirements - Inception

In a commercial off-the-shelf (COTS) application implementation, the “Gather Business Requirements - Inception " activity serves to detail the business and system objectives for the project and begins the process of gathering key business requirements and data needed to configure the application environment.

Approach

In a commercial off-the-shelf (COTS) application implementation, you may have access to pre-existing, multi-level business process models, which depict how the future application system supports the business functions being addressed by the project. If such process models are available, leverage them to help define elements of the Future Process Model (RD.011) and Business Use Case Model (RA.015) prepared during this activity.

A.ACT.CCPI Create Conceptual Prototype - Inception

In a commercial off-the-shelf (COTS) application implementation employing a requirements-driven approach, this activity is generally performed only when there is a need to gain further clarification of the business requirements represented in the use cases. This typically applies to those requirements that can only be satisfied through extensive and/or complex custom-built components identified during the pre-sales cycle and/or during the Inception Phase, which extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

This activity is not normally performed in a commercial-off-the-shelf (COTS) application implementation employing a solution-driven approach that seeks to minimize custom extensions by promoting leading practice use of standard functionality. This activity is therefore not included in the pre-defined Solution-Driven Application Implementation View Work Breakdown Structure. However, if your solution-driven application implementation includes architecturally-significant custom extensions, you should consider including this activity in your project.

Back to Top

ELABORATION

B.ACT.DUCM Develop Use Model
B.ACT.DUCD Develop Use Case Details

In a commercial off-the-shelf (COTS) application implementation employing a requirements-driven approach, Use Case Models complement business process models in helping the implementation team gain a thorough understanding of business requirements. At a minimum, Use Case Models and Use Case Specifications should be

developed for those requirements that can only be satisfied by custom-built components, which extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems. Development of Use Case Models for all other business requirements is considered best practice, as they contribute to a better understanding of requirements and can be leveraged to prepare a number of other required work products downstream, such as test scenarios and business procedures documentation.

In a commercial off-the-shelf (COTS) application implementation employing a solution-driven approach, Use Case Models should be developed only for those requirements that can only be satisfied by custom-built components that extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

B.ACT.AE Analyze - Elaboration

In a commercial off-the-shelf (COTS) application implementation, this activity is generally performed only when there is a need to gain further clarification of the business requirements represented in the use cases. This typically applies to those requirements that can only be satisfied by custom-built components, which extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

B.ACT.DE Design - Elaboration

In a commercial off-the-shelf (COTS) application implementation, this activity is generally performed only when there is a need to gain further clarification of the business requirements represented in the use cases. This typically applies to those requirements that can only be satisfied by custom-built components, which extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

B.ACT.DCCTSE Develop Custom Component Test Scripts - Elaboration

In a commercial off-the-shelf (COTS) application implementation, this activity is performed only for custom-built components that extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

In an application implementation project, unit testing is generally performed in the Construction phase. However, it may be performed in the Elaboration phase for complex, higher risk custom extensions that have been identified during the pre-sales cycle and/or during the Inception phase, and where the prototypes of those complex custom extensions that have been developed during the Elaboration phase, will continue to be enhanced during the Construction phase.

B.ACT.DVCSP Develop and Validate Custom Software Prototypes

In a commercial off-the-shelf (COTS) application implementation, this activity allows for developing and validating the various prototypes of custom extensions to the applications, implementing the database extensions (where necessary) and performing a component review. Development and validation of the Functional Prototype(s) for custom extensions is generally limited to custom extensions that warrant development of a prototype at this stage of the project due to the technical complexity of the extension, or other risks associated with them.

It also serves to address the creation of a User Interface Standards Prototype environment for validation of any revisions made to the out-of-the-box application system user interface in order to meet the implementing organization’s requirements. This is generally applicable only to those COTS application systems, which enable tailoring/revision of the standard user interface via supported means, such as Oracle’s Siebel Tools.

In a commercial-off-the-shelf (COTS) application implementation employing a solution-driven approach that leverages pre-defined setup parameters and/or tools for rapidly establishing a working application environment, use this activity to develop and validate Functional Prototype(s) if there is a need to build functional prototypes of architecturally-significant custom extensions at this stage of the project.

B.ACT.PCCTE Perform Custom Component Testing - Elaboration

In a commercial off-the-shelf (COTS) application implementation, this activity is performed only for custom-built components that extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

In an application implementation project, unit testing is generally performed in the Construction phase. However, it may be performed in the Elaboration phase for complex, higher risk custom extensions that have been identified during the pre-sales cycle and/or during the Inception phase, and where the prototypes of those complex custom extensions that have been developed during the Elaboration phase, will continue to be enhanced during the Construction phase.

B.ACT.PACDE Prepare to Acquire and Convert Data - Elaboration

In a commercial off-the-shelf (COTS) application implementation, this activity consists of producing the standards and completing the high-level (business object level) data mapping needed to confirm that the application software can support the customer organization's business processes and legacy data requirements. Conversion component design and development tasks are executed in a similarly named activity in the Construction phase.

Tasks

The tasks included in the "Prepare to Acquire and Convert Data - Elaboration " activity for Application Implementations are:

ID	Task Name
CV.025	Define Data Acquisition and Conversion Standards
RA.160	Conduct Business Data Source Gap Analysis

Back to Top

CONSTRUCTION

C.ACT.FR Finalize Requirements

In a commercial off-the-shelf (COTS) application implementation, this activity is generally performed only when multiple iterations are planned for the Construction phase. In such cases, this activity is included in the second, and subsequent, iterations.

This activity is not normally performed in a commercial-off-the-shelf (COTS) application implementation employing a solution-driven approach. This task is therefore not included in the pre-defined Solution-Driven Application Implementation View Work Breakdown Structure. However, if your solution-driven application implementation includes two or more iterations within the Construction phase, you should include this task in your project.

C.ACT.AC Analyze - Construction

In a commercial off-the-shelf (COTS) application implementation, this activity is generally performed only when there is a need to gain further clarification of the business requirements represented in the use cases. This typically applies to those requirements that can only be satisfied by custom-built components, which extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

C.ACT.DC Design - Construction

In a commercial off-the-shelf (COTS) application implementation, this activity is generally performed only when there is a need to gain further clarification of the business requirements represented in the use cases. This typically applies to those requirements that can only be satisfied by custom-built components, which extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

C.ACT.PEC Prepare Environments - Construction

In a commercial off-the-shelf (COTS) application implementation, this activity addresses the installation of software environments for various testing, development and documentation purposes.

C.ACT.DCCTSC Develop Custom Component Test Scripts - Construction

In a commercial off-the-shelf (COTS) application implementation, this activity is performed only for custom-built components that extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

In an application implementation project, unit and integration testing is generally performed in the Construction phase. However, they may be performed in the Elaboration phase for complex, higher risk custom extensions that have been identified during the pre-sales cycle and/or during the Inception phase, and where the prototypes of those complex custom extensions that have been developed during the Elaboration phase, will continue to be enhanced during the Construction phase.

C.ACT.ICC Implement Custom Components

In a commercial off-the-shelf (COTS) application implementation, this activity allows for implementing (that is, building) the custom database extensions, custom components, custom services and installation routines needed to extend the capabilities of the application system.

C.ACT.PCCTC Perform Custom Component Testing - Construction

In a commercial off-the-shelf (COTS) application implementation, this activity is performed only for custom-built components that extend the functionality of the COTS system and/or integrate the COTS system with other COTS or legacy systems.

In an application implementation project, unit and integration testing is generally performed in the Construction phase. However, they may be performed in the Elaboration phase for complex, higher risk custom extensions that have been identified during the pre-sales cycle and/or during the Inception phase, and where the prototypes of those complex custom extensions that have been developed during the Elaboration phase, will continue to be enhanced during the Construction phase.

Back to Top

TRANSITION

There are no Application Implementation Supplemental Guidelines for Transition phase activities. These activities should be performed in accordance with the standard guidance.

Back to Top

PRODUCTION

E.ACT.RPP Resolve Production Problems

In a commercial off-the-shelf (COTS) application implementation, the "Resolve Production Problems" activity also involves the execution of the system management procedures, which are performed in perpetuity.

Approach

The ongoing system maintenance begins during this activity. Maintenance is typically not triggered by technical issues that the support or help desk address; this type of maintenance focuses on necessary and preventative actions.

Ongoing system maintenance should address the following:

- timing of maintenance procedures
- reporting unusual occurrences
- interpreting system indicators
- logging completion of maintenance work
- adjusting the software and hardware for anticipated business changes

Verify that all normal, ongoing activities keep the system protected, purged, and operating as efficiently as possible. Make sure a log is available for recording all change events. Standard checklists can be incorporated for normal, standing tasks that must be performed (like backups and defragmentation of tablespace). In addition, use the log as a record when new users are added to the system.

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

APPLICATION IMPLEMENTATION SUPPLEMENTAL TASK GUIDELINES

Back to View

This document contains OUM supplemental task guidance for Application Implementation projects.

- The first column contains the OUM ID.
- The second column of the following table provides a link to the "supplemental" task guidance in this document.
- The third column provides a link to the core OUM guidance.
- The fourth column provides a link to any associated templates, if available.

The guidelines in this document should be used to supplement the guidelines that appear in the core OUM guidance. This index is not a complete list of the activities and tasks for an Application Implementation. It only contains those tasks that have supplemental guidance. Refer to the view for a complete list of tasks.

OUM ID	Implement Supplemental Guidance	OUM Guidance	Template
A	**Inception**	**Inception**	
A.ACT.GBRI	**Gather Business Requirements - Inception**	**Gather Business Requirements - Inception**	
A.RD.012	Document Present and Future Organization Structures	Document Present and Future Organization Structures	Present and Future Organization Structures
A.ACT.GSP	**Gather Supporting Requirements**	**Gather Supporting Requirements**	
A.DO.010	Define Documentation Requirements and Strategy	Define Documentation Requirements and Strategy	Documentation Requirements and Strategy
B	**Elaboration**	**Elaboration**	
B.ACT.DPS	**Define Project Strategy**	**Define Project Strategy**	
B.CV.020	Define Data Acquisition, Conversion and Data Quality Strategy	Define Data Acquisition, Conversion and Data Quality Strategy	Data Acquisition, Conversion and Data Quality Strategy
B.TS.020.1	Define Cutover Strategy	Define Cutover Strategy	Cutover Strategy

B.ACT.DI	**Define Infrastructure**	**Define Infrastructure**	
B.TA.030	Define Integration Requirements and Strategy	Define Integration Requirements and Strategy	Integration Architecture Strategy
B.TA.090	Develop Security and Control Strategy	Develop Security and Control Strategy	Security and Control Strategy
B.ACT.PEE	**Prepare Environments - Elaboration**	**Prepare Environments - Elaboration**	
B.TE.038.1	Prepare Integration Test Environment	Prepare Integration Test Environment	Integration Test Environment
B.ACT.PVC	**Prototype and Validate Configuration**	**Prototype and Validate Configuration**	
B.TE.025.1	Create System Test Scenarios	Create System Test Scenarios	System Test Scenarios
B.ACT.DE	**Design - Elaboration**	**Design - Elaboration**	
B.DS.050	Determine Design and Build Standards	Determine Design and Build Standards	Refer to the Task Overview for guidance.
B.DS.080.1	Design Software Components	Design Software Components	Refer to the Task Overview for guidance.
B.DS.150.1	Develop Database Design	Develop Database Design	Logical Database Design
B.ACT.DCCTSE	**Develop Custom Component Test Scripts - Elaboration**	**Develop Custom Component Test Scripts - Elaboration**	
B.TE.035.1	Create Integration Test Scenarios	Create Integration Test Scenarios	Integration Test Scenarios
B.ACT.DVCSP	**Develop and Validate Custom Software Prototypes**	**Develop and Validate Custom Software Prototypes**	
B.IM.040.1	Implement Database	Implement Database	Physical Database Design
B.ACT.PCCTE	**Perform Custom Component Testing - Elaboration**	**Perform Custom Component Testing - Elaboration**	
B.TE.040.1	Perform Integration Test	Perform Integration Test	Integration Test Results
C	**Construction**	**Construction**	
C.ACT.DC	**Design - Construction**	**Design - Construction**	
C.DS.080.2	Design Software Components	Design Software Components	Refer to the Task Overview for guidance.
C.DS.150.2	Develop Database Design	Develop Database Design	Refer to the Task Overview for guidance.
C.ACT.PEC	**Prepare Environments - Construction**	**Prepare Environments - Construction**	
C.TE.038.2	Prepare Integration Test Environment	Prepare Integration Test Environment	Integration Test Environment
C.ACT.DCCTSC	**Develop Custom Component Test Scripts - Construction**	**Develop Custom Component Test Scripts - Construction**	
C.TE.035.2	Create Integration Test Scenarios	Create Integration Test Scenarios	Integration Test Scenarios
C.ACT.DSTSC	**Develop System Test Scenarios - Construction**	**Develop System Test Scenarios - Construction**	
C.TE.025.3	Create System Test Scenarios	Create System Test Scenarios	System Test Scenarios
C.ACT.ICC	**Implement Custom Components**	**Implement Custom Components**	
C.IM.040.2	Implement Database	Implement Database	Refer to the Task Overview for guidance.

C.ACT.PCCTC	**Perform Custom Component Testing - Construction**	**Perform Custom Component Testing - Construction**	
C.TE.040.2	Perform Integration Test	Perform Integration Test	Integration Test Results
C.ACT.PSTC	**Perform System Test - Construction**	**Perform System Test - Construction**	
C.TE.070.2	Perform System Test	Perform System Test	System Test Results
C.ACT.PSIT	**Perform Systems Integration Test**	**Perform Systems Integration Test**	
C.TE.085	Prepare Systems Integration Test Environment	Prepare Systems Integration Test Environment	Systems Integration Test Environment
C.TE.100	Perform Systems Integration Test	Perform Systems Integration Test	Systems Integration Test Results
C.ACT.PCO	**Prepare for Cutover**	**Prepare for Cutover**	
C.TS.020.2	Define Cutover Strategy	Define Cutover Strategy	Refer to the Task Overview for guidance.
C.ACT.PACDC	**Prepare to Acquire and Convert Data - Construction**	**Prepare to Acquire and Convert Data - Construction**	
C.CV.040.2	Design Conversion Components (Initial Load)	Design Conversion Components (Initial Load)	Conversion Component Designs (Initial Load)
C.ACT.ACDC	**Acquire and Convert Data - Construction**	**Acquire and Convert Data - Construction**	
C.CV.055.2	Implement Conversion Components (Initial Load)	Implement Conversion Components (Initial Load)	Conversion Components (Initial Load)
C.CV.065.1	Convert and Verify Data (Initial Load)	Convert and Verify Data (Initial Load)	Converted and Verified Data (Initial Load)
C.ACT.PD	**Produce Documentation**	**Produce Documentation**	
C.DO.060	Publish User Reference Manual	Publish User Reference Manual	User Reference Manual
C.DO.080	Publish User Guide	Publish Technical Reference Material	Technical Reference Material
D	**Transition**	**Transition**	
D.ACT.CDT	**Convert Data - Transition**	**Convert Data - Transition**	
D.CV.065.2	Convert and Verify Data (Initial Load)	Convert and Verify Data (Initial Load)	Converted and Verified Data (Initial Load)
E	**Production**	**Production**	
E.ACT.RPP	**Resolve Production Problems**	**Resolve Production Problems**	
E.PS.060	Monitor and Respond to System Problems	Monitor and Respond to System Problems	Refer to the Task Overview for guidance.
E.ACT.PF	**Plan for Future**	**Plan for Future**	
E.PS.135	Determine Future Functional Enhancements	Determine Future Functional Enhancements	Refer to the Task Overview for guidance.

Back to Top

INCEPTION

A.ACT.GBRI Gather Business Requirements - Inception

A.RD.012 Document Present and Future Organization Structures

Approach

In a commercial off-the-shelf (COTS) application implementation, the Present and Future Organization Structures should, at a minimum, portray the legal and operating entity structures, business organizations, financial operating environments, revenue and cost centers, and financial consolidation path. It may also catalog other operating entities.

Gathering Organization Information

Information about current and proposed organization structures may have been discussed during the application acquisitions cycle, and may have been one of the deciding factors in the selection of the application package. Be sure to gather this information, if available.

Start the internal organization analysis by interviewing the highest ranking financial official possible, since that position will likely have the most knowledge of the financial and operating structure of the organization.

An organization's operating structure drives the business and has a strong influence on the setup and use of the applications; however, the financial statements will reflect the operating structure to allow profitability, balance sheet and cash flow reporting, and analysis against that structure. Reporting and analysis begins by capturing and valuing operational transactions that occur at the specific event level. These events occur at a specific site that must be properly defined and ultimately set up at the correct organizational level and with the appropriate organizational attributes.

Types of Organization Levels

In your analysis, look for the following types of organization structures:

- Legal Entities - an organization considered as one legal or fiscal entity, based on regulatory or reporting requirements
- Consolidation Entities - these may be of two varieties: a single set of books with a division segment in the general ledger for the consolidation, or two (or more) sets of books consolidated into one parent set of books
- Set of Books - a financial operating environment combining a particular chart of accounts, fiscal calendar, and functional currency
- Operating Units - an operational business unit that uses a common order entry, receivables, payables, or purchasing function
- Reporting Entities - entities that may have different management reporting requirements, depending on classification as revenue, cost, profit, or investment center
- Organizations - a physical or logical site used in manufacturing or distribution that holds inventory or other assets

Financial and Management Reporting Environment

The reporting environment is divided into two categories:

- Financial Reporting - addresses all the reporting requirements of the legal entity
- Management Reporting - based on reporting for the following types of relationships with the parent entity: revenue centers, cost centers, profit centers, and investment centers; management reporting also encompasses operational reporting

Project Impact of Current Financial and Operating Structure Analysis

Once you create a clear picture of the organization structure, you may want to reexamine the project plan for the application implementation, especially in the case of a financial implementation. It may make more sense to implement sequentially by organization, rather than simultaneously across the entire enterprise.

Back to Top

A.ACT.GSP Gather Supporting Requirements

A.DO.010 Define Documentation Requirements and Strategy

Approach

Every project must determine if there is a need to publish project-specific documentation, or whether standard Oracle Commercial Off-the-Shelf (COTS) applications documentation meets their requirements. While most organizations are able to implement Oracle (COTS) applications with standard Oracle documentation, some organizations determine that they need additional, project-specific documentation based on the review of the organization's overall documentation requirements.

During Oracle (COTS) application implementations, interview questions should help you determine if the organization is building any application extensions, whether interfaces to third-party or legacy systems are to be used, and if there are any other compelling reasons for which you may want to build project-specific documentation.

User Reference Manual (DO.060)

In a commercial off-the-shelf (COTS) application implementation, the User Reference Manual explains the functionality of the customizations and extensions to the system users.

Technical Reference Material (DO.080)

In a commercial off-the-shelf (COTS) application implementation, the Technical Reference Manual also describes extensions to any COTS (Commercial Off the Shelf) products. It should supplement the *Oracle Application Technical Reference Manual*.

Back to Top

ELABORATION

B.ACT.DPS Define Project Strategy

B.CV.020 Define Data Acquisition, Conversion and Data Quality Strategy

Task Steps

In an Application Implementation, you have additional task steps AI.1 and AI.2, following task step 12 below:

No.	Task Step	Component	Component Description
1	Provide an introduction for the document.	Introduction	The Introduction describes the work product and how it is used.
2	Document any pertinent background information.	Background	The Background describes how the system is used by your customer management and any other pertinent information you wish to include.
3	Document any pertinent overview information.	Data Acquisition and Conversion Overview	This component provides details on how the Data Acquisition and Conversion process is used to realize the data acquisition, data conversion and data quality requirements of the project.
4	Review the appropriate Technical Architecture (TA) work products.	Data Acquisition - Strategy - Logical Architecture	This component provides the basic information about the logical architecture of the data solution and its components. The main reason why this section is included is to describe the "data stores" that are referenced in the data flow definitions. Note that the term "data store" is used for architectural components of the solution, that are used as a source and/or target of a data flow, i.e., a data store can be a Staging Area, Data Warehouse, Data Mart, OLAP Area, etc.

5	Document source system information for all the data sources identified in the approved Data Acquisition and Conversion Requirements (CV.010).	Data Acquisition - Strategy - Source Systems	This component provides source system information for all the data sources. There may be multiple interfaces from each source system, each interface with different format, frequency and/or content. For the purpose of this component, “interface” is defined as a set of files or tables that are extracted and transferred at the same time using the same extraction and data transfer method.
6	Document the target systems and interfaces to the target system.	Data Acquisition - Strategy - Target Systems	This component of the task documents the target systems. Note there might be multiple interfaces from each source system, each interface with different format, frequency and/or content. If there is no target system (i.e., the data is used for reporting only) then delete this section. For the purpose of this component, “interface” is defined as a set of files or tables that are extracted and transferred at the same time using the same extraction and data transfer method.
7	Analyze the required source and target system interfaces and required business functionality and prepare a list of data flows needed to support the requirements.	Data Acquisition - Strategy - Data Flows	This component includes a diagram that represents data flows. Examples of are provided in the template. Data flow is a process that takes data from a source interface (or data store) and it loads the target data store (or output interface), e.g., a data flow processes data from a particular source system and it loads it into the staging area. Another data flow then loads target data warehouse from the staging area. Data flows described in this section correspond to the processes or jobs executed when running and refreshing the data warehouse.
8	Review the business requirements regarding the initial and historical data in order to document each requirement. Get the sign off from the customer for the initial and historical load requirements.	Data Acquisition - Data Load Strategy - Initial and Historical Loads	The Initial and Historical Loads component provides a table listing the requirements for the initial or historical loads from source systems. It is important to note for each requirement how many months (days) of data have to be loaded, whether the historical data is available, what is the expected volume of data and whether it is possible to load the historical data with the processes developed for regular updates.
9	Provide guidelines for raising errors, notifying administrators and enabling error recovery.	Data Acquisition - Data Load Strategy - Error Handling Strategy	This component describes strategies for raising errors, notifying administrators and recovery steps after the error is resolved.
10	Review the Data Acquisition and Conversion Requirements, specifically the Data Conversion sections.	Data Conversion - Requirements	This component lists the legacy system entities being converted.
11	Review the Data Acquisition and Conversion Requirements, specifically the Data Conversion sections.	Data Conversion - Constraints and Assumptions	This component lists any constraints and assumptions on which the tasks, resources, and conversion execution strategies are based.
12	Develop a simple, effective and quick approach to convert only the data needed to provide business benefits in the initial production period.	Data Conversion - Conversion Approach	The Conversion Approach describes the general approach that will be used to guide the conversion portion of the Data Acquisition and Conversion process.
AI.1	Prepare Conversion Process Flows for each target application to which you are	Data Conversion -	This component documents the conversion process flow for each target application to which you are converting data. These process flows document related application business object prerequisites as

	converting legacy data.	Process Flows	well as the conversion sequence of master and transactional business objects.
AI.2	Describe the high-level conversion testing strategy.	Data Conversion - Testing Strategy	This component describes the procedures to follow when testing the conversion programs and testing how the converted data functions in the target application. An example of a testing procedure is the comparison of record counts between the legacy and the target application.
13	Review the source systems and Data Quality components of the Data Acquisition and Conversion Requirements (CV.010). Define the objectives of data quality in detail.	Data Quality - Objectives	This component defines the objectives of the data quality management in the solution. The following areas are described in this component: • Data Quality Overview • Data Quality Measures • Data Quality Objectives
14	Review the source systems and Data Quality components of the Data Acquisition and Conversion Requirements (CV.010). Provide the detail requirements for the Data Quality processes.	Data Quality - Processes	This component provides the description of the processes related to the data quality. The following processes will be designed in such a way in order for the solution to meet the data quality objectives. • Data Transfer • Data Cleansing • Audit and Control • Error and Exception Handling
15	Review the source systems and Data Quality components of the Data Acquisition and Conversion Requirements (CV.010). Provide the detail requirements for the Data Quality ownership.	Data Quality - Data Ownership	This component defines the ownership of data quality components and responsibility for solving end escalating data quality related issues. Include the following sections. • Data Ownership • Issue Resolution • Escalation Hierarchy
16	Document the transformation rules in a table.	Data Quality - Transformation Details	This component defines the data quality transformations.

Approach

Data Conversion

When discussing the conversion approach, you should consider the number of conversion cycles that will be performed before the final conversion. At a minimum, you should complete the full conversion testing cycle and use converted legacy data during Perform System Test (TE.070.2). Your goal is to determine the length of time it will take to convert all of the legacy data to the target applications before the final cutover to production. To set expectations correctly, include the conversion programs in Performance Management (PT) testing.

Automated Conversion Tools

In addition to traditional coding tools, consider including automated conversion tools when producing the conversion approach component of this deliverable. Using these tools has several advantages:

- The automated conversion tools reduce the need for traditional programming skills.
- The automated conversion tools can be used to build reusable templates for multi-site implementations.

When employing automated conversion tools, the content of the Conversion Program Designs (CV.040) and the functionality of the resulting Conversion Components (CV.055) can vary depending on the conversion tool you use.

Data Cleansing

Decide whether to perform data cleanup in the legacy system prior to conversion or to clean up the legacy data using the target application environment after loading the data into destination tables. An example of data that must be cleaned up is a customer that has been entered multiple times in slightly different ways. XYZ Corporation, XYZ Co. and Xyz Company may all represent the same customer entity, but due to the difference in the way these values were entered (varying use of upper/lower/mixed case, terminology and abbreviations) they are treated as separate entities by the system.

Back to Top

B.TS.020.1 Define Cutover Strategy

Approach

Transition

The general sequence for transition is:

- Prepare the Production Environment.
- Configure (set up) the applications.
- Execute the Conversion Programs.
- Register modifications.
- Verify conversion data and application setups.
- Plan the transition according to these guidelines:
- Set up the communications and computing hardware.
- Install the software applications, tools, and custom modules.
- Prepare the environments.
- Configure the application environments.
- Integrate the applications.
- Convert the data.
- Verify the data and application configuration.
- Run the reconciliation.

Support Infrastructure

Allocate time in the transition plan and schedule for system administrators or key project team members to implement the support infrastructure. This includes development and rollout of a help desk learning event for information systems support personnel, as well as users.

The establishment of a robust and capable internal and vendor external support infrastructure is critical to the success of your project. Oracle Support may be of assistance in helping you assess our current support capabilities and assist with planning, updating, and aligning these capabilities to Oracle's support infrastructure.

Vendor Awareness of Cutover

The transition period may make extra demands on the support services of the hardware, network, and software vendors. By giving the vendors advance warning of the production cutover, you can alert them to possible extra demands on support services and make them aware of the critical nature of the business situation.

Process Teams

Separate functional and setup responsibilities by process team. Project team members that have responsibility for the design of new business processes will also be responsible for the execution and validation of the application configuration and converted data in the production system.

Process teams verify the setup for the applications and responsibilities, and the integration of custom modules, reports, and inquiry screens. If some data is to be manually converted, the process teams can either divide the data entry work or subcontract a temporary employee to enter the data. The latter case requires temporary staff

learning events as well as an additional audit of the data by the process team members.

System Administration

The earlier that system administrators assume their new role in supporting the system, the better. You should prepare a checklist for all system administration responsibilities including form and program registration, custom menu definition, application user accounts, integration of non-Oracle applications, and descriptive and key flexfield setup. Create a system administration checklist for all the tasks and activities that involve execution of the production system.

Contingency Planning

Planning for contingencies in the migration process requires, as a first step, identification of the risks and failure points in the process. Having identified these areas, you can outline a high-level course of action for handling each eventuality. Because of the focus on risks and failure points, this task is tightly linked to Verify Production Readiness (TS.058). The detailed analysis that is used in Verify Production Readiness can also be used here, at a higher level, for identifying general risk areas. The identification of contingency measures may be unpopular because it has connotations of project failure and mishandling, but a more thorough contingency plan can potentially reduce the amount and degree of crisis management needed should the unthinkable happen. Do not confuse implementation contingency with system contingency failure.

Back to Top

B.ACT.DI Define Infrastructure

B.TA.030 Define Integration Requirements and Strategy

Approach

The primary purpose of this task is to map distributed data external integration points between Oracle commercial off-the-shelf (COTS) applications and other third party or legacy applications. Once you agree on an alternative for each integration point gap, this task will result in a complete listing of all new integration points for which you must design and build interfaces. At this stage there has been no specification of interfaces; this task is merely a recognition that an integration point exists and requires attention.

External Integration Points

The most common type of integration point requiring attention in an implementation project is one that transfers data between two heterogeneous applications. External integration points are also needed between two separate installations of the same application with separate databases. These integration points are external in nature since they result in the transfer of data from application databases external to the one of interest. They usually give rise to interfaces that support the integration.

Replacement External Integration Points

The simplest way to identify the new external integration points is to perform a thorough architecture baseline analysis, then identify the changes implied and required by the replacement of certain existing legacy applications. The business requirement for each old interface should be met in some way by the new system, either as a direct mapping, or as a change in implementation or functionality.

Although the approach helps scope and validate the completeness of the mapping work, it also carries significant risks. It requires significant attention to details when you define the existing technical architecture, and it does not guarantee that new interface requirements will be noticed and documented. The technique carries the risk of re-implementing outmoded practices, rather than leveraging the capabilities of new applications, and may miss critical requirements that would only become apparent with an understanding of the role of the interface in one or more Use Cases (RA.023). By using a tangible mapping of old interfaces to new, you may find it easier to focus on specific implementation work products.

Nonstandard External Integration Points

If you are implementing a combination of custom developed applications together with Oracle COTS applications, also consider new integration points that you need to

handle between your new, nonstandard applications and legacy applications that you are planning to retain in the new architecture. These other new integration points will be revealed by performing the replacement mapping not just for replacement by Oracle COTS applications, but for replacement by your entire new applications suite. Overlooking these integration points is another risk of the “cut out the old and replace with the new” method of interface development.

Integration Points from Standard Applications Distributed Data

If your implementation spans separate, standard applications databases, it is critical to identify the external interfaces that need to cross databases. This will always be the case for multi-site installations with multiple data centers. These types of interfaces can be more complex to implement, and you should not attempt to perform requirements gathering for them using this type of bottom-up analysis. However, even if you are managing the development separately, you can still record the integration points as part of this task so that you have a complete reference document for all integration points to the new applications.

Back to Top

B.TA.090 Develop Security and Control Strategy

Approach

User Function or Role-Based Security

This type of security is oriented towards the functions, transactions, forms, or reports to which a user can gain access. This approach to security secures data by preventing users from gaining access to any data in a particular table or from performing a particular business function.

Business Organization-Based Security

This type of security analysis is oriented towards securing the data for a particular business organization or user group. Unlike the function- or role-based security, this area typically deals with securing data for organizations as a whole and not just for individuals or roles within an organization. Often, business organizations may be creating rows in the same tables in a database, but do not wish other organizations to be able to view or transact their data in the same table. The division between this approach to security and the function/role-based approach is not hard and fast, and it may be possible to satisfy the business organization-based security requirements by judicious use of function/role-based security and the standard application features that support it.

Designing Business Organization-Based Security

The design of solutions for business organization-based security is sometimes more difficult than the function/role-based security, because it affects the storage and partitioning of data in the database. The application functional architecture and key setup parameters may have an effect on the security and partitioning of data also. The approach you should take to satisfy a particular organizational requirement is detailed below. Whether one solution is more complex than another will depend on the exact circumstances for the implementation project and the business itself:

- Use standard application features that provide organization-based security.
- Use standard application function/role-based security to provide *the effect of* business organization-based security.
- Use procedures and policies to provide the security.
- Use limited custom extensions to application modules.
- Use general database-level custom extensions.
- Use separate application installations.

Many security requirements may be met using standard application features. Before exploring more complex options, be sure to identify the security requirements that can be met by standard application security features for both function/role-based security and organization-based security. The cost and extra risk of custom security mechanisms may force a rethink of whether a particular security requirement is really needed.

Use of Policies and Procedures

In many cases, business procedures and policies are an effective and much simpler approach to securing data than designing system-level controls, especially when the required controls are not readily available through functionality built into the applications suite you are implementing. System auditing also falls into this category of

approach. Determine whether security across organizations must be enforced by system controls, before embarking on design activities.

Back to Top

B.ACT.PEE Prepare Environments - Elaboration

B.TE.038.1 Prepare Integration Test Environment

Work Product

In a commercial off-the-shelf (COTS) application implementation, setting up an environment includes the configuration, or parameter setting, of the environment as well. In addition, patching may also be required. It is important that all project environments be kept in sync both from a set up point of view as well as from a patching level point of view.

Approach

Environment Configuration

Configure the Integration Test Environment based on the latest application setups from the Application Setup Documents (MC.050). Before you begin testing in each environment, you should:

1. Set up the applications.
2. Decide on the real business data to use.
3. Plan the volume of data (for example, converted data).
4. Verify that all application parameters have been set up to enable transactions.
5. Verify that sufficient business data has been entered to demonstrate application features.
6. Provide access to definition and setup screens to appropriate users.
7. Enable links to non-Oracle or custom systems (if applicable).
8. Make reporting, data query, and testing tools available in the testing environments to verify correct results against the expected results of the test scripts.
9. Record all changes and updates to the test environment to help you maintain the integrity of the objects that have been installed or updated.

Cloning

In the case of packaged application(s), the environment can also be prepared by means of cloning techniques. This avoids time consuming manual set up activities.

Import/Export

If you have entered your setups into a master setup environment, you may choose to export that data and import it into your test environments. If testing uncovers issues that affect setups, you must change the setups in your master environment so that they will be correct for production.

Back to Top

B.ACT.PVC Prototype and Validate Configuration

B.TE.025.1 Create System Test Scenarios

Approach

Documenting the Test Scenarios

Use the System Test Scenario to document the steps needed to test the operation and integration of business system flows within the (COTS) application system, including the integration of application extensions.

System Testing

System testing measures the quality of the entire application system, using system test sequences and scenarios. You must create scripts for all business processes based on the Future Process Model (RD.011) and Use Case Specification (RA.024). The system test can include the following types of testing:

- integrated business processes
- manual procedures
- support procedures
- security testing
- initial system documentation inspection
- manual data inspection
- database journaling
- converted data load testing
- converted data inspection
- interface testing (limited to processing data as input from another system, or creating data for use by another system)
- data/transaction reconciliation to the legacy system
- job stream testing (if there is a batch component)
- backup and recovery testing
- data archival testing
- lock testing simulating user load (executing identical scenarios)
- batch window test (on full converted data volumes)

Back to Top

B.ACT.DE Design - Elaboration

B.DS.050 Determine Design and Build Standards

Approach

Should you not find any satisfactory standards and guidelines, it is recommended that another separate project be initiated to produce acceptable standards and guidelines.

The Design and Build Standards allow you to document customizations in a way that clearly communicates the features and functionality to both users and developers. They also help ensure that new modules have a consistent look and feel and integrate well with the standard Oracle COTS applications.

Each standard you define should provide a specific design, development, documentation, or learning benefit. Good standards help you do the following:

- Explain functionality to current and potential users.
- Train users.
- Support users when they have questions or problems.
- Manage the development of customizations.
- Produce high-quality program modules.
- Work efficiently and cost-effectively.
- Port application modules to different hardware and software platforms.
- Upgrade customizations to new releases of the applications.

Do not duplicate what Oracle has already defined in standard documentation. Cosmetic standards help ensure the same look and feel of the applications. To users, maintaining the same format and style is important. Use Oracle design and development standards to maintain the same appearance of new or modified applications. These standards not only provide cosmetic guidelines, but development standards for building custom modules as well.

Back to Top

B.DS.080.1 Design Software Components

Approach

Upgrades Design

Unlike standalone custom applications, extensions to packaged applications must be designed so that they can be easily migrated to future releases of the base product. Fortunately, Oracle E-Business Suite includes several features that facilitate upgrades.

Configurable Extensions

Configurable extensions such as flexfields, and alerts are automatically retained when you upgrade to new application releases . Leverage these features when designing customizations. For example, instead of modifying a form by adding a new section, define a new form with the required information and build a link between the two forms. Use descriptive flexfields whenever you need additional data elements, instead of modifying standard tables.

Custom ORACLE IDs

An ORACLE ID identifies a registered database user that is in Application Object Library. Each standard Oracle Application has a corresponding ORACLE ID. Define all custom tables in an ORACLE ID you create for your custom application.

Use a prefix (such as CSTM or an appropriate acronym for your organization) to distinguish custom ORACLE IDs and tables from standard ones. For example, if you create a custom Purchasing form that updates a custom table, you would create the table in the CSTMPO ORACLE ID and name the table CSTMPO_HEADER_DETAILS.

Grants and synonyms allow your forms and other custom programs to access the custom table.

Back to Top

B.DS.150.1 Develop Database Design

Work Product

Use the Logical Database Design when you must capture and document both the logical data representation and the physical design of new database objects. This work product should address the following:

- the relationships between new business objects and existing application entities
- the design of new database tables and related views, grants and synonyms
- indexing requirements
- tablespace and sizing requirements
- flexfield designs

Approach

The Logical Database Design defines changes to the objects in the database prior to designing the individual modules that make up all of the defined customizations. In reality, it is often a distributed activity where individual designers define the database extension requirements as they are writing the Software Components Design

(DS.080) work product . However, it is still important to assign one database designer to maintain the overall data model.

Because Oracle COTS applications ship with a predefined database model, this task is limited to new database objects needed to implement required functionality. If you do not require new database objects, the scope of this document is limited to flexfield design.

Integration with Standard Tables

When you add tables to implement new business objects and entities, it is important to show how those new tables relate to the existing database schema on an entity-relationship diagram. Use a unique box style to clearly distinguish new from existing entities.

Since you should not modify existing tables, you may need to add new tables, even if your requirement is to add attributes (columns) to an existing entity. The new table will have a one-to-one relationship with the existing table and use the same unique identifier columns. The only time this should be a requirement is when descriptive flexfields cannot accommodate the required attributes.

Descriptive Flexfields

Descriptive flexfields are a type of database extension that does not require any new tables or columns. However, because several modules may use flexfields on the same table for different reasons, it is important to have a single point of contact to manage the flexfield definitions for application entities.

Coordinate with business analysts who need to define flexfield setup parameters as part of Define Application Setups (MC.050).

In a multi-site implementation, this activity is very important because each site may have different requirements for descriptive flexfields and you may need a single definition that satisfies all requirements. You may need a global data registry to fully satisfy your requirements.

Back to Top

B.ACT.DCCTSE Develop Custom Component Test Scripts - Elaboration

B.TE.035.1 Create Integration Test Scenarios

Approach

Documenting the Test Scenarios

Use the Integration Test Scenarios to check the linkage and integration between two or more application extension components that are part of the same business process. You create one Integration Test Scenario for each application extension, which may consist of several individual application extension components (such as a program, form, report, flexfield, database trigger, or other application extension components).

Integration Test Scenario Test Data

If the Integration Test Environment is a fresh installation, you need to construct the test data or use converted data from preliminary conversion programs. If the Integration Test Environment is a copy of another environment, you should design your test data based on information already defined in the database.

Business Flow Testing

Business flow testing involves developing the Integration Test Scenarios to test the business processes that include the customization. Business flow testing is the primary and most important form of integration testing. The Software Components Design work product (DS.080), Use Case Specifications (RA.024) and Future Process Model (RD.011) provide a solid basis for developing the test steps and data to test the business flow.

Your tests should also include unusual or extreme business flows, even if they are highly unlikely. Experience has shown that users can and will perform the most unlikely procedures.

Integrity Testing

Any application extension that updates data must be able to handle record locks. Part of your integration test should evaluate how the application extensions respond to this condition. Appropriate design strategies include waiting for locks to be released or simply aborting when a program cannot obtain a lock.

Batch Programs

A common error in batch programs is selecting data from tables without a lock at the start of processing, and then updating the original tables with new data at the end. In this scenario, another program can update the table between the original data selection and the final update, resulting in data loss (the middle transaction is lost). This can cause severe referential integrity problems when the middle transaction updates other tables with synchronized data that is not lost.

You can designate concurrent programs as incompatible with themselves and other concurrent programs that manipulate data from the same tables. This can prevent locking problems caused by two update programs running simultaneously. The Software Components Design work product (DS.080) should define this requirement and you should configure the test environment accordingly. Test to make sure that the concurrent manager enforces the rules appropriately.

Testing Techniques

Use these techniques to evaluate lock integrity:

Forms

When two users attempt to update the same record via a form, the first user to make a change should secure a lock while the other user is asked if they want to wait for the lock to be released. Sloppy programming can cause forms to engage a lock when a user simply queries a record, and can even result in a deadlock situation where neither user can release the lock.

Use two sessions to query the same row into identical forms by different users. Attempt to have both users lock the row by modifying a data element. Verify that the form handles the lock properly and releases it when you save the change or clear the form (commit or rollback).

Make sure that the form does not inadvertently lock the rows of query-only forms or blocks. Query the row on one form while using an update form for the base table (or SQL*Plus) to test the lock condition.

Concurrent Programs

Run two copies of the same concurrent program simultaneously. One instance should wait for the other to complete or abort with an appropriate error message.

Run the concurrent program and use a form to lock a row that the program will update. The program should wait a reasonable amount of time or abort with an appropriate message.

Run the concurrent program and use a form to lock a row that the program reads but does not update. The program should complete without any errors.

Run the concurrent program and use a form or SQL script to update a row that the program also updates. If you receive a message that the form or SQL*Plus cannot obtain a lock, make sure the program releases its lock when it completes. If the transaction succeeds, verify that both sets of updates are preserved.

Usability Testing

Usability testing does not focus on the accuracy of an application extension, but on user-friendliness. Naturally, users are the best resource for this type of testing; they should be able to use the application extensions as part of a business flow after reviewing the Use Case Specification (RA.024) and receiving basic instructions.

If you perform this type of testing, your role is to observe the user’s actions and responses to the software. Take notes to document functionality or program behavior that confuses the user or elicits an incorrect response.

Usability testing is a way of refining the design of an application extension and is common when following an iterative design and build approach to custom development. Consider usability testing whenever the user interface or procedures are complex or unusual.

Security Testing

Security testing involves two areas: user access to application extensions (menus), and application extension access to data (including data residing in both owned and shared tables). You must verify that each user has access to the correct responsibilities, menus, and data to perform his or her job. You must also verify that each application extension has access to both its owned tables as well as any shared tables.

Volume Testing

Volume testing involves simulating live operations with multiple users active on the same program, as well as on conflicting programs. Have multiple users access combinations of the same application extensions and test data while performing the same operations to evaluate any obvious conflicts or performance issues. Coordinate any volume testing with the Performance Testing team on your project.

Back to Top

B.ACT.DVCSP Develop and Validate Custom Software Prototypes

B.IM.040.1 Implement Database

Approach

Separate ORACLE IDs

Create custom database objects in custom ORACLE IDs (an Oracle user name registered within Application Object Library) so that custom objects are not mixed with standard database objects.

Grants and Synonyms

Include scripts that create the database grants and synonyms required to allow other ORACLE IDs to access the custom tables. Since custom tables, views, indexes, and sequences exist in a custom ORACLE ID, you need grants and synonyms to allow custom forms and concurrent programs to access the tables.

Back to Top

B.ACT.PCCTE Perform Custom Component Testing - Elaboration

B.TE.040.1 Perform Integration Test

Approach

Integration-tested modules are the result of performing integration testing on custom application extension components. The name *integration test* comes from testing the connections, or *integration,* between application extension components. Integration testing should uncover all potential problems with the custom application extension code.

Document Test Results

Record actual results, specific data entered, and other comments in the *actual results* column of the Integration Test Results (TE.040). You can also include screen shots and sample report output to support your results.

Integration testing verifies that all components of a custom application extension work together properly and integrate seamlessly with standard application modules. For example, an application extension to support unique purchasing requirements may include a new database table, a new form, and a new report. Each or these custom components would have been tested individually during Perform Unit Test (TE.030). The integration test employs a business scenario to test the end-to-end flow of all of

the custom application extension components together.

The integration test also extends the test to include other standard Oracle COTS application modules that either precede or follow the steps in the flow that exercise the application extensions. Integration testing should incorporate functions that rely on data manipulated by the application extensions.

Build up to the full integration test by testing programs that work together—first in pairs, then in sets of three, four, and so on. Test simple scenarios first and then build more complex cases with planned exceptions to test warning and error trapping. Finally, test the customization in the context of other related application functions.

Although the already developed Integration Test Scenarios (TE.035) primarily dictate your integration testing process, you may think of other variations and scenarios that you should test. Test these new scenarios and document each one in a new integration Test Results (TE.040).

Back to Top

CONSTRUCTION

C.ACT.DC Design - Construction

C.DS.080.2 Design Software Components

See guidance for B.DS.080.1.

Back to Top

C.DS.150.2 Develop Database Design

See guidance for B.DS.150.1.

Back to Top

C.ACT.PEC Prepare Environments - Construction

C.TE.038.2 Prepare Integration Test Environment

See guidance for B.TE.038.1.

Back to Top

C.ACT.DCCTSC Develop Custom Component Test Scripts - Construction

C.TE.035.2 Create Integration Test Scenarios

See guidance for B.TE.035.1.

Back to Top

C.ACT.DSTSC Develop System Test Scenarios - Construction

C.TE.025.3 Create System Test Scenarios

See guidance for B.TE.025.1.

Back to Top

C.ACT.ICC Implement Custom Components

C.IM.040.2 Implement Database

See guidance for B.IM.040.1.

Back to Top

C.ACT.PCCTC Perform Custom Component Testing - Construction

C.TE.040.2 Perform Integration Test

See guidance for B.TE.040.1.

Back to Top

C.ACT.PSTC Perform System Test - Construction

C.TE.070.2 Perform System Test

Approach

During Perform System Test, users develop confidence in the programs and the overall usability of the target application system. It is important to test operating routines and procedures as well as computer programs, thereby simulating business operations, not just systems.

The repetitive nature of this task allows the project team to test each business flow and incrementally improve the system until they reach the desired level of success. Test each process flow until you achieve success. Then integrate the script with each other across process boundaries.

Successful business system testing should verify the following:

- the system successfully supports business operations
- all results are predictable and repeatable
- variations of the test script produces expected results
- data is verifiable through alternative means across the business system
- relevant key performance indicators can be validated
- support procedures intended for production are accurate
- modifications to procedures, policy, and system are fully implemented

Relationship to Configuration Prototype Validation

Conduct Configuration Prototyping Workshop (MC.080) and Perform System Test are similar tasks—they are both based on business processes and they both employ the use of test scripts and their components. They differ, however, in the test data used, the level of applications setup validation, and in the user skills and level of documentation required to thoroughly execute the tests.

System testing involves validation of converted data, whereas functional prototype validation uses manually keyed sample data, because generally converted data is not available at that time. System testing also validates the integration of the entire suite of application configuration setups that were individually tested during functional prototype validation. The final iteration of system testing includes all custom application extensions that are not available (perhaps not even designed) during functional prototype validation. System testing, however, does not include testing interfaces to legacy and third-party systems. These interfaces, if they exist, are tested during Perform Systems Integration Test (TE.100).

The skills required for the system testing team are also similar to those required for functional prototype validation, except that system testing requires more discipline with regard to recording details (actual results), while functional prototype validation involves validating that a given process can be supported. System testing also tends to be more comprehensive from the standpoint that tests repeat iteratively, and for all action items resulting from the tests, you must resolve, retest, and document any changes to application setup for implementation in the production environment.

Organizing Testing Teams

Organize system testing teams by assigning specific job roles to individual testers. This prevents testers from inappropriately carrying forward knowledge of past test steps that another role would not possess. Key users should assume roles that match their actual jobs. They should be familiar with both the Oracle COTS application modules and custom application extension functionality, as well as the testing process itself.

Always organize testing sessions for efficiency by publishing a thorough agenda that includes:

- session location and duration
- role assignments
- tests to be executed
- activities and sequence
- props and other physical setups that help simulate realism
- expected outputs and criteria for successful closure of the testing session

Role Playing

Use role playing and props to simulate business processes. For example, when a purchasing agent prints a purchase order, have it delivered to the *vendor*, who responds by shipping products and *mailing* an invoice. Furthermore, try putting the purchase order into a window envelope. This might reveal that the printed address does not display completely through the window—a subtlety that could easily be overlooked with more casual testing. Even though actions such as these may only involve passing paper across a table, they are intended to mimic real life as much as possible in an attempt to flush out potential problems with the system.

Keep verbal communication to a minimum, except when it is truly part of the process. For example, avoid the temptation to say to the shipping clerk, “I have just loaded some orders into the system for a Z500-32X configuration. Try shipping them. ”The process and test plans should dictate when the shipping clerk checks for orders and picks products to ship. The spoken communication would not take place in real life.

Parallel Process Flows

Where processes overlap or are independent, it is possible to perform process testing in parallel with coordination. For example, processing purchase orders and building items in Work in Process (WIP) can be performed independently. The only coordination is at the points when the processes intersect, such as outside processing of WIP operations or receiving material on a purchase order directly to WIP.

Follow Documented User Procedures

The System Test Scenarios (TE.025) should be based on user procedures. The documented user procedures should also be available and used for reference (just as they would be available at each user’s desk when the new system is in place).

Use Converted Data

Convert all or a part of the legacy system data for the system test. If you cannot convert all legacy data for the test, convert a representative sample. For example, if you are converting General Ledger balances, make sure that you include converted journals from each cost center, business unit, division, and so on. If you are converting vendors, make sure that the data includes vendors with multiple sites and multiple contacts, so that you can test full vendor-related functionality and conversion. Keep in mind that the system test serves as the final test of the conversion programs prior to transition.

Multiple Copies of Test Scenarios

Typically, you will execute the system test more than once. After completing a test execution (iteration), you should have a set of detailed and summary test results that have been recorded in the System Test Scenarios (TE.025). Between iterations you correct defects found during testing and prepare for the next iteration. When the next iteration is complete, you can compare results from each iteration to measure quality gains or losses, and use this information to manage quality and plan for additional test iterations, if necessary.

The System Test Scenarios (TE.025) include blank columns to record actual results and resolution notes. Keep a master copy of the scenarios with these blank columns so you can make additional copies for repeated tests. Fill out the results by hand on the forms and file multiple copies of the same test together with an identifying iteration or cycle number. At the completion of the test, update the master soft copy with the final test result notes for each test step.

Regression Testing

The regression test retests changes made to a modified application extension because of an enhancement or a correction to a coding error. The regression test gets its name from the principle that when the development of a system progresses, validation of that progression requires proof that all prior validations have not regressed.

In regression testing, you will retest application extensions in order to validate that prior application extension defects have been corrected. The regression test generally reexecutes the entire Unit Test Script (TE.020) and a selection of related Integration Test Scenarios (TE.035) to confirm that the overall quality of the software has improved. You perform regression testing in order to answer the questions:

- How do I prove that the defect has been fixed?
- How do I prove that it did not cause another defect somewhere else?

In development environments where libraries of shared objects exist and object sharing is encouraged, be careful to run *where used* reports that expose all application extensions affected by a defect in a common object. Your regression test will need to select test scripts that retest all of the application extensions that invoke the common module.

Determine whether your regression test will run concurrently with system, systems integration, and acceptance tests. Some projects use the regression test to stabilize application defects for the next iteration of the system test. This strategy allows the current system test to proceed to completion while the regression test stabilizes and fixes defects, thus preparing for the next iteration of the system test. Other projects use regression testing as the final test after fixing all faults.

Automated Regression Testing

Determine your level of regression testing automation. Regression test automation is now a very realistic option. Current tools feature the ability to record user actions and document GUI objects, thereby capturing this information into test scripts. You can then generalize the scripts to use variable data, enabling the script execution to be data-driven. If your project is considering automating its regression testing, clearly state your automation goals and be sure to plan extra time for script development and generalization. Test scripts now become a baselined item and you must take care to update the scripts each time the application is enhanced, due to approved change control, scope increase, and so on.

Document the extent of automation you expect to achieve by leveraging regression testing tools. Be careful not to overlook the overhead of automation, as you will need additional development time and skill to perform automated regression testing.

Recreating the Test Conditions

When attempting to validate that a defect has been fixed, be careful to recreate the same data conditions and sequence of events that exposed the original defect.

Converted Data

When testing with converted data, verify that the data was not the cause of the problem. Application extensions are usually designed to handle data conditions created by the application system they are part of. If the application extensions were not built with an emphasis on error-handling, introducing converted data can cause application extensions to fail.

Follow Support Procedures

When you encounter problems, use the Production Support Infrastructure Design (TS.040) as a guide to test the organization's new internal support procedures. When these procedures do not resolve the problem, call Oracle Support (or other external support) and log the technical issue with them.

Back to Top

C.ACT.PSIT Perform Systems Integration Test

C.TE.085 Prepare Systems Integration Test Environment

Approach

Environment Configuration

Configure the Systems Integration Test Environment based on the latest application setups from the Application Setup Documents (MC.050). Before you begin testing in each environment, you should:

1. Set up the applications.
2. Decide on the real business data to use.
3. Plan the volume of data (for example, converted data).
4. Verify that all application parameters have been set up to enable transactions.
5. Verify that sufficient business data has been entered to demonstrate application features.
6. Provide access to definition and setup screens to appropriate users.
7. Enable links to non-Oracle or custom systems (if applicable).
8. Make reporting, data query, and testing tools available in the testing environments to verify correct results against the expected results of the test scripts.
9. Record all changes and updates to the test environment to help you maintain the integrity of the objects that have been installed or updated.

Import/Export

If you have entered your setups into a master setup environment, you may choose to export that data and import it into your test environments. If testing uncovers issues that affect setups, you must change the setups in your master environment so that they will be correct for production.

Back to Top

C.TE.100 Perform Systems Integration Test

Approach

The purpose of the systems integration test is to test the operation of the business system across and between application systems. Use this task to develop user confidence in the overall usability of the system, including interfaces to external systems. It is important to test operating routines and procedures as well as computer programs, thereby simulating business operations, not just systems.

Successful systems integration testing should verify the following:

- The integration points (interfaces) can successfully support business operations across and between application systems.
- All integration test results are predictable and repeatable.
- Variations of the test scripts produce expected results.
- Data are verifiable through alternative means across the business systems.
- The support procedures intended for production are accurate.
- Modifications to the procedures, policy, and system are fully implemented.

Back to Top

C.ACT.PCO Prepare for Cutover

C.TS.020.2 Define Cutover Strategy

See guidance for B.TS.020.1.

Back to Top

C.ACT.PACDC Prepare to Acquire and Convert Data - Construction

C.CV.040.2 Design Conversion Components (Initial Load)

Approach

In general, the approach and technique you use for designing your conversion programs is impacted by the method you choose for coding your conversion programs. During the conversion process, you may use traditional coding methods, using tools such as PL/SQL and SQL*Loader, or automated conversion tools. Regardless of the methods you choose for coding your conversion programs, you must determine where to document the conversion program design.

Download/Extract Programs

Typically, the organization's information systems support staff is responsible for providing extract files for conversion testing and conversion production loading. However, depending on the tool that you are using to upload the data to Oracle, you may have to be very specific about how you want the extract file to look. An example of the logic that you need to document is: *There must be multiple records per order in the extract file for order lines.*

Depending on the conversion requirements, the data extraction may be performed multiple times. If this is the case, you may want to consider using one of Oracle's gateway products to provide direct legacy system access.

Interface Tables

During conversion, do not directly populate Oracle COTS applications tables. The legacy data should first be moved to one or more of the standard or custom-developed interface tables. The number of interface tables required for the conversion of a business object depends on the complexity of the business object being converted. The level of complexity is not necessarily driven by data volume, but is more dependent on the number of processing, translation, filter, and foreign key rules. Many of the Oracle COTS applications have standard APIs that provide interface tables to load the legacy data before executing the interface programs.

You can write traditional SQL code to create these tables, or if you are using an automated conversion tool, you can extend the Oracle production tables to include processing columns for processing rules. The temporary tables provide a place for you to apply all of the rules you have defined for a specific business object, before moving the data into the production tables.

Uploading Data

There are many tools available for uploading the data to the Oracle tables. The interface tables should already exist (in the case of a standard API) or should have been created before the upload module is run. Other loader options include the following:

- Use the loader facilities offered as standard functionality in automated conversion tools.
- Upload legacy data directly from a spreadsheet or PC database to the Oracle database tables using available ODBC drivers and SQL*Net, etc.
- Translation Programs

Once you create and load the interface tables, create and execute special logic to manipulate the data to attain the correct format, which the new application needs to operate. Execute this data manipulation before moving the data to the Oracle production tables. You can write traditional code using tools such as SQL*Plus or PL/SQL or use an automated conversion tool.

Interface/Validation Programs

Typically, the level of validation required must be consistent with the validation being performed by any form or package logic. Therefore, it is important to remember that these interface programs not only move the data to the production tables but also perform all required data validation. For example, if the order entry form validates that the customer on an order is already a customer that exists in the order entry system, then your interface module should do the same. If the level of validation is not the same as the form-level validation, when the converted order is queried in the new Oracle system, the user may get an error message.

An automated conversion tool may also be the best option for writing your interface modules. The tool can be used to perform the necessary lookups required to enforce data validation in other Oracle tables.

Back to Top

C.ACT.ACDC Acquire and Convert Data - Construction

C.CV.055.2 Implement Conversion Components (Initial Load)

Approach

Code the conversion programs following the build standards for custom extensions and any supplemental standards defined specifically for conversion activities in the Data Acquisition and Conversion Standards (CV.025). In many projects, customer staff members may perform the final conversion, and therefore may be executing the modules created by a development team member. In this case, provide the customer staff members with a conversion execution document that details what each module does and the order in which the modules should be run to achieve a successful conversion. Use checklists to document conversion prerequisites and post-conversion activities.

As mentioned in the Conversion Component Designs (CV.040), many Oracle Applications have standard interfaces that facilitate the conversion coding effort. In addition, consider using automated conversion tools instead of traditional coding methods. Below are some approaches and techniques for each of the five conversion programming areas.

Download Programs

The customer staff members are normally responsible for writing and maintaining the download modules. As with the Conversion Component Designs (CV.040), it is important that you provide them with a file schema that describes how many ASCII files to produce for a given business object, the order of the fields, the delimiter to be used, and so on. Depending on the tool you are using for the conversion and the legacy file structure, you may want to combine data in one of the ways listed below:

- single extract file to populate multiple Oracle tables
- multiple passes of a single extract file to populate multiple Oracle tables
- multiple extract files to populate multiple Oracle tables
- multiple passes of multiple extract files to populate multiple Oracle tables
- single extract files to populate a single Oracle table
- multiple passes of a single extract file to populate a single Oracle table

This decision is largely dependent on the relationship and complexity of the Oracle tables being populated.

Interface Table Creation Programs

Use SQL*Plus to create the interface tables in Oracle. You can build an interface table for each Oracle production table you are going to load. The interface table can be a direct copy of the production table, or can have only the columns within the production table that you are going to load.

You do not want the interface table to contain the constraints that the production tables have. This is because you are performing the data translation in the interface tables prior to validation and insertion. Therefore, if you copy the production table to use as your interface table, you probably want to remove the constraints, grants, and synonyms. For example, if the extract file contains a Y but the database is expecting a numeric value, then you may load the Y into the temporary table and translate the Y to a numeric value of 1 before passing the value to the production table.

If the Oracle application has a standard API to facilitate the conversion of a given business object, then you do not have to create an interface table. In this case, the Oracle application provides an interface table that you load the legacy data elements and data defaults into. If however, there is a great deal of data translation that needs to take place before the legacy data is loaded into the Oracle applications interface tables, then you may want to create an additional translation table. Refer to the *Oracle Applications Open Interfaces Manual* for specific information on how to populate the interface table columns, as well as information on how to execute the concurrent process.

Automated Conversion Tools

If you are using an automated conversion tool, you may be able to use the tool to create the interface tables. Most of the logic for data validation, transformation, and so on may also be built into the tool templates. However, you may need to create interface tables for mapping between old and new data values. Refer to the documentation of the tool you are using for information on its capabilities and use.

Upload Programs

Use a loader tool such as SQL*Loader to load the legacy system data flat file into the temporary tables you have created. If you are using an automated conversion tool, it may provide this functionality. Automated conversion tools allow you to map the legacy data to the Oracle table and columns, perform the required translations and validations, and then load the data into Oracle.

Automated Conversion Tools

If you are using an automated conversion tool, you may be able to define all the validation, transformation, and translation rules in the tool. Refer to the documentation of the tool in use for information on its capabilities and use.

Translation Programs

Create the translation programs using an appropriate programming language, depending on the complexity of the translations and your skills. SQL and PL/SQL are the preferred tools for creating the translation programs. The level of complexity of the translation depends on the legacy system being converted. Is the legacy system normalized? Is new functionality being built into Oracle that was not used in the legacy system? These types of questions directly affect the level of complexity of the translation programs.

Some or most of the data translation can be built on the legacy side, depending on available IS support resources. The Conversion Data Mapping (CV.027) and Conversion Component Designs (CV.040) should provide the developer with all of the necessary logic for the data translations. Some examples of translations follow:

- date formatting
- truncation of values
- concatenation of values
- if/then logic

As mentioned previously, automated conversion tools are an option for building and executing this code.

Interface/Validation Programs

The complexity of the interface modules is primarily determined by the level of validation required for the data being converted. To guarantee that the data is useable in the new Oracle Application, your interface programs need to perform the same level of validation as the forms of the application. For example, if a form validates that a sales person assigned to a customer must already exist in the system, your interface/validation program needs to perform the same level of validation. Perform a lookup against the table that stores the sales person information to confirm the existence of such data.

If the Oracle application provides a standard API, execute the interface program as a concurrent process to provide the required level of validation.

The interface modules are typically written in PL/SQL or Pro*C. However, there are several automated tools that build and execute the code to validate and load the production tables.

If you are using an automated conversion tool, you may be able to build most of the validation logic in the tool. Refer to the documentation of the tool you are using use for information on it's capabilities and use.

Back to Top

C.CV.065.1 Convert and Verify Data (Initial Load)

Approach

Achieve Converted and Verified Data by successfully executing the conversion programs to perform the following functions:

- Download the data from the legacy system and create a flat file that can be transferred and uploaded to Oracle tables.
- Create Oracle interface tables that can store the legacy data before the data is validated and moved to the production tables of the Oracle application.
- Load the legacy data to the temporary tables.
- Perform any required data translation, transformation, or cleanup before moving the data to the production tables.
- Interface (validate) and move the data to the Oracle production tables.

Back to Top

C.ACT.PD Produce Documentation

C.DO.060 Publish User Reference Manual

Work Product

In a commercial off-the-shelf (COTS) application implementation, use the User Reference Manual to explain to users in functional terms how the application extensions will work.

Audience, Distribution and Usage

In a commercial off-the-shelf (COTS) application implementation, the User Reference Manual is used to show users how the application extensions enhance standard Oracle (COTS) applications functionality (use the preliminary versions).

In addition, the User Reference Manual includes information about custom forms and reports and is an extension of the standard reference manuals provided with Oracle (COTS) applications. Describe each visible title, field, option, and transaction type for any custom software or application extension.

Back to Top

C.DO.080 Publish Technical Reference Material

Work Product

In a commercial off-the-shelf (COTS) application implementation, the Technical Reference Material provides technical information regarding application extensions and interfaces.

Approach

The Technical Reference Documentation does not replace the standard Oracle COTS application product technical reference manuals, so there is no need to duplicate information found in the standard Oracle COTS application product technical reference manuals . Although this manual is directed at technicians, remember that there can be a wide range of knowledge and experience within the technical group.

Audience, Distribution and Usage

For custom extensions and interfaces to the Oracle (COTS) application products, the content of the document should supplement the standard Oracle COTS application product technical reference manuals.

Back to Top

TRANSITION

D.ACT.CDT Convert Data - Transition

D.CV.065.2 Convert and Verify Data (Initial Load)

See guidance for C.CV.065.1.

Back to Top

PRODUCTION

E.ACT.RPP Resolve Production Problems

E.PS.060 Monitor and Respond to System Problems

Approach

In a commercial off-the-shelf (COTS) application implementation, the ongoing system maintenance begins during this task. Create a system or process to effectively maintain the production system and respond to users' requests.

Routine Maintenance

Routine maintenance is performed by the information systems operations group. Routine maintenance includes:

- running batch reports
- running extracts
- logging file purges

enlarging tablespace
- monitoring table space defragmentation
- running hot backups
- running cold backups
- performing security audits
- runaway process management
- performance tuning
- archiving and purging data

On Demand Maintenance

On demand maintenance includes unplanned or requested activities such as the following:

- setting up new users
- changing new user access
- correcting interface data
- correcting data due to user error

Back to Top

E.ACT.PF Plan for Future

E.PS.135 Determine Future Functional Enhancements

Approach

Future Enhancements Collection During the Implementation

The content for the Future Functional Enhancements can be collected during the Business Requirements (RD), Design (DS) and Implement (IM) activities. Luxury or wish list features may take an extraordinary amount of project team time to produce. These are captured in a separate document to be considered for a subsequent implementation, and should be categorized so that management can determine the urgency of need, estimate the effort required, and justify the enhancements.

Future Applications Releases Upgrade

Looking forward, you should consider the new features planned for the applications. Consultants should be involved with this process. If you have insight with regard to what the next release of the applications provide, then you should recommend or highlight these as a justification to upgrade.

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

ORACLE USER PRODUCTIVITY KIT PROFESSIONAL (UPK PRO) AND ORACLE TUTOR SUPPLEMENTAL GUIDE

The materials presented in this tool guideline provide supplemental guidance for using Oracle® User Productivity Kit Professional (UPK Pro) and Oracle® Tutor in conjunction with OUM. Oracle Tutor provides users with tools to develop relevant process documentation and learning materials. Oracle UPK provides tools for annotated transaction simulations depicting proper field-by-field execution of key system processes. UPK Pro Knowledge Center provides an easy-to-search knowledge portal that provides stakeholders, project managers and employees with access to a central repository of all project assets. Used with available baseline content, the materials developed by these tools are process and role-based, easy to update, cost-effective and developed in conjunction with the implementation to accurately depict the future state model. Implementation documentation developed with UPK Pro and Tutor can be leveraged and used to create training content to enable end users to be productive at go-live.

The Oracle UPK Pro and Oracle Tutor Supplemental Guide contains the following components:

Oracle UPK Pro and Oracle Tutor Overview

This component provides an overview of Oracle UPK Pro and Oracle Tutor.

Oracle UPK Pro and Oracle Tutor Supplemental Task Guidelines

The Supplemental Task Guidelines should be used in conjunction with the standard OUM tasks when using UPK Pro and Tutor tools.

Oracle UPK and Oracle Tutor White Papers

For more information on UPK and Tutor, review white papers found on oracle.com:

UPK White Papers

Tutor White Papers

PDF View

You may also prefer to access this information through the **Oracle UPK Pro and Oracle Tutor Supplemental Guide (PDF View)** located in the Supplemental Guides section of the Supplemental Guidance page.

Back to Top

PURPOSE

This document contains OUM supplemental task guidance for using User Productivity Kit Professional (UPK Pro) and Oracle Tutor in conjunction with OUM. UPK Pro and Tutor are complementary Oracle tools that are used to develop and maintain content that can be used throughout each phase of the enterprise application implementation lifecycle from blueprinting/design/testing to go-live and maintenance/support. UPK Pro and Tutor can be used with all of Oracle's business application brands as well as non-Oracle business applications. UPK Pro and Tutor allow project teams to streamline the creation of business process documentation, test scripts, interactive simulations, job aids, presentations, courseware and in-application performance support reflecting the specific business practices and applications environment. For creating the above content types, use Tutor for business process documentation and UPK Pro for everything else.

Back to Top

AUDIENCE

This document is intended for practitioners who will be using Oracle UPK Professional and Oracle Tutor while designing and implementing custom solutions based on the OUM (Oracle Unified Method) approach.

Back to Top

HOW TO USE THIS DOCUMENT

This document contains supplemental guidance for using UPK Pro and Tutor in conjunction with and not instead of the OUM base method guidance.

Back to Top

LIMITATIONS TO THIS DOCUMENT

This document does not describe OUM in detail – the reader must review the documentation and training materials referenced below to gain a basic understanding of each.

Back to Top

RELATED DOCUMENTATION AND TRAINING

The following related documentation & training provide additional detailed guidance related to OUM implementations and are prerequisites to this document:

- OUM Method Pack – Desktop Installable or Hosted version
- OUM Training and Accreditation Program

Back to Top

CONTRIBUTORS

Emily Chorba, Chuck Jones

Back to Top

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

ORACLE USER PRODUCTIVITY KIT PROFESSIONAL (UPK PRO) AND ORACLE TUTOR OVERVIEW

TOOL OVERVIEW

Oracle User Productivity Kit Professional (UPK Pro) and Oracle Tutor are complementary Oracle tools that are used to develop, deploy and maintain content that can be used throughout each phase of the enterprise application implementation lifecycle from blueprinting/design/testing to go-live and maintenance and support. UPK Pro and Tutor can be used with all of Oracle's business application brands as well as non-Oracle business applications. UPK Pro and Tutor allow project teams to streamline the creation of business process documentation, test scripts, interactive simulations, job aids, presentations, courseware, and in-application performance support, reflecting the specific business practices and applications environment. For creating the above content types, use Tutor for business process documentation and UPK for everything else.

UPK Pro's collaborative content development platform is used by the project Testing, Functional, Training and Sustaining teams to very quickly and efficiently create and maintain synchronized System Documentation, Test Scripts, Transaction Process Simulations, and in-application end-user support. Prebuilt UPK baseline content is available for sale to customers, and not only enables a rapid start on simulation development, but can be used in a functional prototype without requiring a test instance of the application to be set up. The UPK software can create over a dozen assets from one recording effort. As part of online training, users can use the simulation playback to be trained and tested on their understanding of how to complete activities that are part of their daily job.

Tutor provides content and software tools to easily create, modify, and maintain business process documentation - policies, procedures, and supporting documents. Author is a modification to MS Word, and is therefore easy to learn and use. The documentation created with Tutor depicts both the business process flow, as well as the supporting business procedure level. Tutor Author contains automated flowchart generation that makes the creation of flowcharts simple and easy. Tutor Publisher creates html based desk manuals, produces QA reports, and generates the complete html set in batch. Finally, Tutor Model Documents contain over 250 baseline procedures that enable a rapid start to developing end-user documentation.

Both UPK and Tutor content can be deployed on the organization's intranet, and in an Oracle Applications Unlimited instance. When deployed with an application, the end user can access content based on the transaction they are executing via the Help button. This provides superior in-production support to the end-user community.

Knowledge Center provides an easy-to-search knowledge portal that provides stakeholders, project managers and employees with access to a central repository of all project assets. All published UPK content, project documentation and Tutor procedures can be deployed to present a guided learning path focused on organizational roles.

By documenting the application data entry steps with UPK, employees can be provided with the exact information they need at the exact point they need it. By documenting processes and procedures with Tutor, implementing organizations are also able to provide transparency of their business practices to any other concerned audience (executive staff, internal auditors, external auditors, customers) and for formal regulatory requirements such as ISO9000, Sarbanes-Oxley, HIPPA, and SAS70.

Back to Top

END-USER DOCUMENTATION OVERVIEW

A successful end user documentation project requires a comprehensive plan, and Oracle has developed the End-User Documentation method for Tutor, UPK and Knowledge Center, combining long-standing UPK and Tutor Methods with the structure of Oracle Unified Method (OUM), and integrating the development of UPK Pro deliverables. The resulting approach can be integrated into an existing applications implementation project that is utilizing OUM. In addition, it can be used as a guide for a stand-alone documentation project.

End-user documentation is a critical factor in the success of applications implementations projects. A crucial project event is the development of an effective “future state model” by the project team. This future state model is a carefully defined prototype, which enables the business software to be configured to work most effectively for the implementing organization. It is crucial to communicate this to-be model to the employees within the implementing organization who will be using the applications to perform their work once the system goes live. Oracle UPK Pro and Oracle Tutor, along with this method, provide an effective means to address this challenge and prepare the end-user community to be productive on day one of go-live.

It is strongly recommended that users of this document read and become familiar with The Tutor Solution, the first chapter in the *Tutor Implementation Guide*.

Project End-User Documentation Work Products

Oracle UPK Pro and Oracle Tutor provide the following categories of documentation work products that can be used throughout the lifecycle of an application implementation. The primary work products are:

- **Blueprinting/Design/Testing** - Capture best practices early and use presentation outputs for visual comprehension and review with subject matter experts (SMEs). Use system process documents do document process flows. Use test scripts to validate in conference room pilot. Gain key LOB/locations/Users buy in. Use "See it" mode with voice explanations. Walk through process flows using presentation outputs or system process documents.
- **Business Process Documentation** - Tutor Procedures are role-based work instructions that guide the end user in the proper use of business systems in their organization. Procedures are deployed as html and can be linked to from within the application environment, creating a living set of business process documentation easily accessible to the end-user community. Tutor also provides a hierarchy of process flows and sub processes that drill down into the procedures. The UPK documents are MS Word-based outputs of transaction recordings, and include system process and test documents.
- **Test Plans** - With UPK, capture test steps automatically and them import those steps directly into Oracle Application Testing Suite, IBM Rational or HP Quality Center testing suites eliminating many of the errors that occur when writing manual tests. These test scripts, along with system process and test documents, decrease the testing time and costs during conference room pilots and shorten the project timeline.
- **Transaction Recordings** - UPK develops annotated transaction recordings that depict how Oracle Applications can be executed on a field-by-field basis. The implementing organization develops the transaction recordings to show exactly how a user would perform a transaction in their environment. MS Word-based system process documents, test documents, job aids, instructor manuals and training guides are co-products of the transaction recordings. Like procedures, transaction recordings can be linked to from the application environment, and procedures and transaction recordings can link to each other.
- **Oracle Applications Help Documents** - Oracle Applications contain Help documentation deployed as context sensitive html. When an end user is in a specific transaction and presses the help button, a help document opens that applies to that transaction. Oracle UPK transaction recording content can be integrated directly into the application help system, providing end users context-sensitive help and just-in-time support and information.
- **Courseware** - The project content work products created with UPK can be leveraged to create self-paced or classroom training materials for instructor-led training (ILT).
- **Guided Training Portals** - Knowledge Center, a component of UPK Pro, creates an easy-to-search knowledge portal that provides stakeholders, project managers and employees with access to a central repository of all project assets.

The following table shows the relationship between Tutor and UPK key output documents and OUM tasks and associated work products:

Tutor and UPK Document	OUM Work Products and Tasks
Tutor Business Flow (REnnnnnn)	Current and Future Process Model (RD.011.1, RD.030, RD.011.2)

Tutor Procedure (PRnnnnnn)	Business Procedure Documents (RA.055)
UPK Test Case and Test Documents	Develop Unit Test Scripts (TE.020.1 and TE.020.2) Perform Integration Test (TE.040.1 and TE.040.20
UPK Simulation Topic	User Learningware (TR.080)
Tutor Desk Manual	User Guide (DO.070)
UPK Training or Instructor Guides	User Learningware (TR.080)

The UPK Prebuilt content and the outputs can supplement tasks and project work products in most of the implementation processes (from RA to PS).

Project teams can download baseline content for UPK and Tutor to use as a starting point, then customize it to map to the intended use of the application in the specific organizational environment. Contact your Oracle representative for information on access and pricing for this content.

Documentation Project Resource Requirements

Documentation projects require specialized expertise and dedicated resources. The tools are easy to learn, although not all resources in a project need to learn the complete tool set. The following resources are needed for such a project. Project teams may be composed of resources from consulting organizations, from the customer organization, and from third-party contractors hired for the effort. The customer organization is advised to take on as many of these execution roles as resources allow, ensuring a higher degree of success in the long term.

- **Executive Sponsor** - Customer executive providing vision and authority
- **Project Manager** - Project manager from the consulting organization
- **Customer Project Manager** - Project manager from the customer organization
- **Applications Specialist** - Oracle Applications functional expert
- **Tutor & UPK Consultant** – Tutor and/or UPK experts from the consulting organization or Oracle University
- **Document Owner** - Subject matter expert from the customer user community who has the authority to determine how a process operates, and the responsibility to make sure it continues to meet business needs today and into the future
- **Document Controller** - Tutor Publisher expert who manages QA and publishing
- **Document Specialist** - Tutor Author expert
- **UPK Developer** - Oracle UPK expert or SME
- **Developer** - Create test scripts with UPK
- **Business Analyst** - Create test scripts with UPK
- **Tester** - Conduct tests using with UPK scripts and documents
- **Course Designer** - Knowledge Center development expert
- **Instructor** - Oracle University Tutor or UPK expert
- **Department Manager** - Customer manager whose organization is being impacted by change
- **Auditor** - External or internal auditor

Back to Top

REFERENCES

The following documents are baseline material used in preparing for a documentation project.

UPK Pro Reference Manuals

UPK Pro Reference Manuals are installed with UPK and are located on the Programs>User Productivity Kit>Documentation menu. Select the Reference folder.

Content Development
- Content Deployment
- In Applications Support
- Upgrade
- Installation and Administration
- Knowledge Center: User and Content Management
- Knowledge Center: Reports Management

UPK Prebuilt Content

- Prebuilt UPK content can be purchased in product sets that map to Oracle Applications Unlimited Products.

Tutor User Manuals

User Manuals are installed with the Tutor products. Once Tutor has been downloaded and installed, Manuals can be located in the Tutor\User Manuals directory.

- Oracle Tutor Installation Guide
- Tutor Implementation Guide
- Tutor Author User Manual
- Procedure Style Guide
- Publisher User Manual
- Tutor Document Register [DOCREG.XLS]

Tutor Content

Supplemental Tutor content is installed with the Tutor model documents. Once the Tutor model documents patch has been downloaded and installed, this content can be located in the Tutori\US\Orig directories \INS or \REF.

- Tutor Glossary of Terms [REF0008Y]
- Job Titles by Department [REF0009Y]
- Instructions for developing Tutor Content [INSnnnnY]

Training

Contact Oracle University for information regarding relevant courses.

For More Information

Contact oraclesales_us@oracle.com for information about acquiring UPK.

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

ORACLE USER PRODUCTIVITY KIT PROFESSIONAL (UPK PRO) AND ORACLE TUTOR SUPPLEMENTAL TASK GUIDELINES

This document contains OUM supplemental task guidance for UPK Pro and Tutor.

- The first column of the following table provides a complete list of all tasks in this document for which "supplemental" guidelines are provided as well as a quick link to the "supplemental" task guideline in this document.
- The second column provides a link to the OUM Task Overview.

The task guidelines in this document should be used to supplement the guidelines that appear in the OUM Task Overview.

UPK Pro and Tutor - OUM Supplemental Task Guidelines (this document) collapse table | expand table	OUM Task Overview
Business Requirements	
RD.001 Detail Business and System Objectives	RD.001 Detail Business and System Objectives
RD.011.1 Develop Future Process Model	RD.011 Develop Future Process Model
RD.030 Develop Current Business Process Model	RD.030 Develop Current Business Process Model
RD.070 Determine Audit and Control Requirements	RD.070 Determine Audit and Control Requirements
RD.011.2 Develop Future Process Model	RD.011 Develop Future Process Model
Requirements Analysis	
RA.055 Document Business Procedures	RA.055 Document Business Procedures
Documentation	
D0.010 Define Documentation Requirements and Strategy	D0.010 Define Documentation Requirements and Strategy
DO.020 Define Documentation Standards and Procedures	DO.020 Define Documentation Standards and Procedures
DO.040 Prepare Documentation Environment	DO.040 Prepare Documentation Environment

DO.070 Publish User Guide	DO.070 Publish User Guide
DO.100 Produce Online Help	DO.100 Produce Online Help
Organizational Change Management	
OCM.140 Develop Communication Campaign	OCM.140 Develop Communication Campaign
OCM.180 Determine Impact of Job Changes	OCM.180 Determine Impact of Job Changes
OCM.210 Conduct Managers' Unit/Department Impact Workshop	OCM.210 Conduct Managers' Unit/Department Impact Workshop
Training	
TR.020 Prepare Project Team Learning Plan	TR.020 Prepare Project Team Learning Plan
TR.050 Conduct Project Team Learning Events	TR.050 Conduct Project Team Learning Events
TR.080 Develop User Learningware	TR.080 Develop User Learningware
TR.090 Prepare User Learning Environment	TR.090 Prepare User Learning Environment
TR.100 Conduct User Learning Events	TR.100 Conduct User Learning Events
Operations and Support	
PS.010 Audit System	PS.010 Audit System

Back to Top

BUSINESS REQUIREMENTS (RD) GUIDANCE

RD.001 - Detail Business and System Objectives

Developing an accurate future state model is a critical activity in an applications implementation. Although the future state model may meet the organizations needs precisely, the only way the end user community can be immediately productive and meet the overall business objectives is through accurate user documentation and effective training. The to-be model must be effectively communicated to the end user community.

In order to achieve project objectives and value realization associated with an implementation, it is critical for key executives to understand, support, and communicate the importance of documentation and training. Change management strategy should be defined at the executive level, including vision, risk mitigation, approach, resource utilization, and process management:

- The scope of the business processes being documented.
- The scope of the customer organization being covered including locations and network.
- Potential risks related to the degree of document ownership and customer involvement.
- Documentation project management.
- Documentation resources, including implementers, consultants, project team members, and end users.
- Scope of documentation work products.
- Availability and source of prebuilt content.
- Coordination of documentation development with other efforts, including applications implementations or compliance activity. Will procedures and transaction recordings be developed during functional prototyping activities or later? If later, what documents will be used to communicate the defined future state model to the documentation effort?
- Document management approach, including central server vs single user considerations. Will the UPK environment be collaborative and server based or single user?

Documentation and content deployment tools available for use, and how they will work together.

RD.011.1 - Develop Future Process Model

Oracle Tutor has the capability to create multi-level business flow diagrams, which map to OUM Process Model Documents. The benefit of using Tutor Author to create these documents is Tutor's capability to automatically generate a flowchart from a task-based list of processes. While the process list can contain detailed information about the process in the form of notes, the flowchart is simply the process diagram.

The Tutor hierarchy starts with the process, then drills down to the sub process, then drills down to the procedure. The linking is defined during the document creation process, and once the html version of the content is generated from Tutor Author or Publisher, all the links become active.

Below is a representation of the work product - a process list in Tutor Author. Note the following:

- Decisions are provided via 'If…Otherwise…' statements in Author, which create diamond branches in the flowchart.
- Drill down to Detailed Processes is made possible by the links in the gray bar lines.

Sub process

1. Analyze to Agreement

Sub process

2. Requisition to Receipt

Sub process

3. Inspection to Disposition

If material passes inspection, goto task #6.

If Supplier will be providing a replacement, goto task #5.

If returning material to supplier, goto task #4.

Sub process

4. Supplier Return to Debit

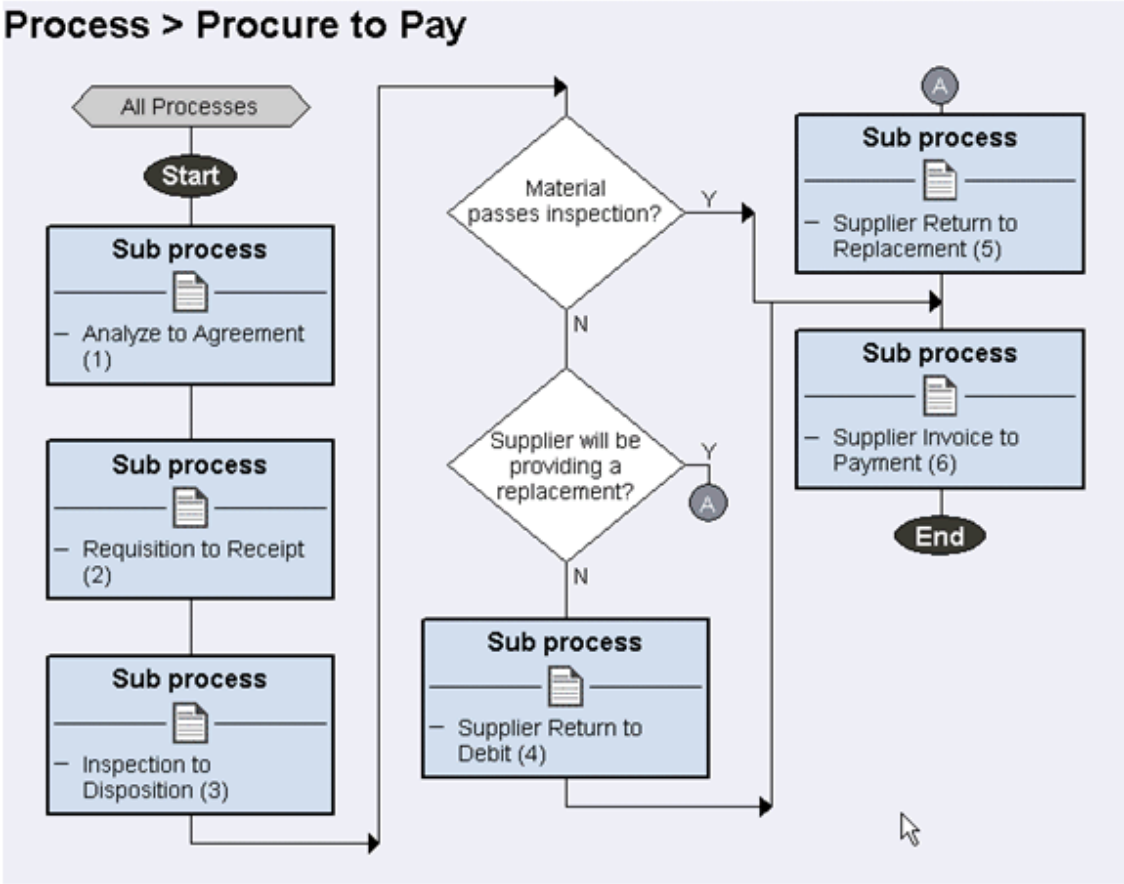
Goto task #6.

Sub process

5. Supplier Return to Replacement

Tutor Process in Text Format

Below is the flowchart of the above process list depicting the complete Procure to Pay process. The Tutor Author's one-click flowchart generation means revising the flowcharts becomes very easy.



Tutor Process in Diagram Format

The Tutor template for the process Model Document is the Process.doc, and is made available when Tutor Author is initiated from the desktop.

For detailed information about building a detailed process model with Tutor see the Tutor Instruction INS1076Y, Writing a Process or Sub Process Map. It is in the following directory when Tutor is installed on the desktop:

C:\Tutor\US\Orig\Ins

RD.030 - Develop Current Business Process Model

The process for creating current state and future state models is the same. See RD.011.1 above and RD.011.2 below for more information.

RD.070 - Determine Audit and Control Requirements

Implementing organizations need to communicate effectively to employees what is expected of them as they perform their jobs. Documented end-user procedures are the best way to communicate job performance expectations and are a significant part of a company's readiness to meet new standards for corporate governance. Integration of business procedures and how well they are supported by cohesive applications merits serious consideration. While documented procedures alone cannot enforce corporate governance, they are a necessary component of compliance. Increasing the visibility of, and improving the control of your business practices through

documentation will better prepare your organization for the remainder of the Sarbanes-Oxley attestation audit process. Documented business procedures are the first step in corporate governance.

Oracle Tutor's integrated set of procedures and software tools offer companies a solution to quickly document, deploy, and maintain critical business procedures and training materials to help employees achieve compliance. The primary challenge for companies faced with documenting procedures is to realize that they can do their corporate governance documentation in-house, with existing resources, using Oracle Tutor.

RD.011.2 - Develop Future Process Model

Oracle Tutor has the capability to create multi-level business flow diagrams, which map to OUM Process Model Documents. The benefit of using Tutor Author to create these documents is Tutor's capability to automatically generate a flowchart from a task-based list of processes. While the process list can contain detailed information about the process in the form of notes, the flowchart is simply the process diagram.

The Tutor hierarchy starts with the process, then drills down to the sub process, then drills down to the procedure. The linking is defined during the document creation process, and once the html version of the content is generated from Tutor Author or Publisher, all the links become active.

Below is a representation of the work product - a detailed sub process model in list form in Tutor Author. Note the following:

- Prior Activity depicts processes which precede the current process.
- Decisions are provided via 'If…Otherwise…' statements in Author that create diamond branches in the flowchart.
- Drill down to Processes is made possible by the links in the colored bar lines.
- Annotations are possible for the processes.

Procedure

1. Requesting Material and Services.

Procedure

2. Selecting a Supplier.

If new vendor required, goto task #3. Otherwise, goto task #5.

Procedure

3. Evaluating Supplier Performance.

Procedure

4. Adding Suppliers, as needed.

Procedure

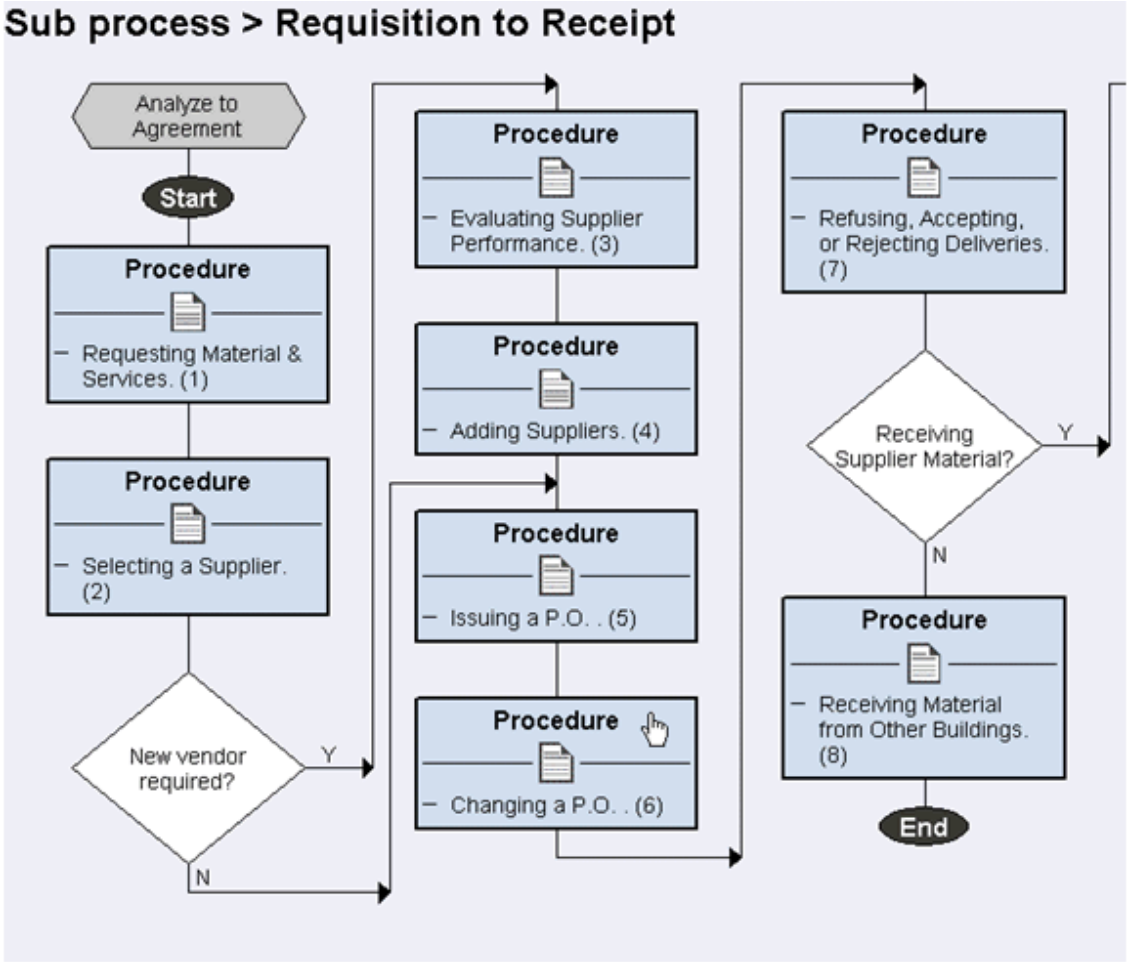
5. Issuing a Purchase Order.

Procedure

6. Changing a Purchase Order, as needed.

Tutor Sub Process in Text Format

Below is the flowchart of the above sub process list. Tutor Author's one-click flowchart generation means revising the flowcharts becomes very easy. When this flowchart is presented in its html format, all hexagonal and rectangular objects contain links created automatically by Tutor.



Tutor Sub Process in Diagram Format

The Tutor template for the process Model Document is the Sub Process.doc, and is made available when Tutor Author is initiated from the desktop.

For detailed information about building a detailed process model with Tutor, see the Tutor Instruction INS1076Y, Writing a Process and Sub Process Map. It is in the following directory when Tutor is installed on the desktop:

C:\Tutor\US\Orig\Ins

Back to Top

REQUIREMENTS ANALYSIS (RA) PROCESS GUIDANCE

RA.055 Document Business Procedures

Oracle Tutor's integrated set of procedures and software tools offer implementing organizations a solution to quickly document, deploy, and maintain critical business procedures and reference materials to help employees achieve compliance. Oracle Tutor is comprised of the following components:

- Tutor Model documents are complete documents that provide a starting point. Document “owners” edit the models instead of creating documents from scratch.
- Tutor Software tools are used to keep documents up-to-date and to distribute them on a need-to-know basis.
    - Author, with Microsoft Word, is the tool used to create and edit individual documents quickly and easily.
    - Publisher is the tool used to (1) ensure document integrity through the generation of a wide range of cross-reference reports, and (2) package process documents in desk manuals (online or printed) by job title.

No.	Task Step	Description
1	Conduct Procedure Editing Workshop.	This workshop may be an extension of a previous team Learning Event, but It is very useful to develop the first set of procedures in a group environment with an instructor who can provide answers to frequently asked questions, and critique the initial set of revised procedures. This leads to more uniform content produced by the entire team. This workshop can last from 1-2 days up to 2-4 weeks, depending on the project environment. In some cases, the procedure editing may also be a component of one of the functional prototyping activities or workshops. The document owner works with the applications specialist to develop the proper combination of policy, business, and application content. If Model Documents are being used, it may be appropriate to initiate editing by conducting a redline editing session. In this case, paper copies of the procedures are distributed and team members modify them with a red pen. Documentation specialists then make the changes in the softcopy documents and redistribute. As part of this process, the group may want to create a list of valid job titles in the organization. Refer to *Job Titles by Department [REF0009Y].*
2	Edit Procedures.	Once the workshop has run its course, the documentation team develops the remaining procedures. Best practice is to conduct this editing during the functional prototyping activities or workshops. This makes capturing the future state model more effective. Other options are to use use cases or test scripts as a source of content, but this can lead to inaccurate procedures if there is no effective review loop with the project team. It is wise to have the team members post their content daily to Oracle Files, or another repository, for protection. See the following Tutor Documents for additional guidance: • INS1066Y, Reviewing Model Documents • INS1076Y, Writing a Process or Sub Process Map • INS1050Y, Writing a Procedure • INS1068Y, Review Edited or Audited Procedures • INS1070Y, Update a Procedure • INS1061Y, Capturing Application Help Targets • INS1065Y, Capturing Screen Images • INS1051Y, Writing an Instruction • INS1057Y, Writing a Reference Document • INS1080Y, Link a Procedure to a UPK Help Topic

3	QA Procedures.	QA is managed in two ways:
		
		• Compare modified procedures with the standards list and templates to ensure compliance.
		• The Document Controller runs all documents through Tutor Publisher and prints out key reports. The reports will identify anomalies such as inaccurately named roles, which can be fed back to the procedure development team for correction. See the Reports section in the Tutor Publisher User Manual for a description of the reports. Key report usage and issues include:
		  ◦ ACR2SCRN: Actor to Screen Report can be used to validate the accuracy of job role titles and access to key transactions.
		  ◦ DISCDOC: Discrepant Document Report is used to ensure that all cross referencing of other procedures is accurate.
		  ◦ POL2PROC: Policy to Procedure Report identifies all policies (indicated by bold text in the policy section) referenced for guiding behavior in the procedures.
		
		The QA cycle is performed until the content meets standards. For the verification of complete two-way referencing of procedures to Help documents, the QA must be coordinated with the Help Document QA process.

A Tutor Procedure contains three sections.

- Introduction section contains scope, policy, distribution, responsibility, ownership, activity preface, and prior activity sections.
- Task List section contains role based tasks with decisions, notes, and system references.
- Flowchart section contains the flowchart diagram of the task list.

Note in the complete example of a work product below there is no system reference:

Petty Cash Requests

Quick Link Petty Cash Requests Go

Scope

This procedure covers petty cash requests.

This procedure does not cover employee expense reports. Refer to Approving Employee Expense Reports.

Policy

An **expense reimbursement** from petty cash must be

- less than $150
- supported by a receipt (or equivalent) that documents the nature of the expense

Responsibility

The Accounting Clerk is responsible for

- maintaining the petty cash box
- ensuring that expense reimbursements from petty cash meet company policy

Distribution

Accounting Clerk*

Employee*

Ownership

The Controller <e-mail> is responsible for ensuring that this document is necessary, reflects actual practice, and supports corporate policy.

Activity Preface

Procedure Introduction Section

Employee

1. Submit the receipt for the expense to the Accounting Clerk responsible for petty cash.

- Record the nature of the expense on the receipt if it is not already evident.

Accounting Clerk

2. Determine whether the expense meets company policy for reimbursement from petty cash.

   The expense must be less than $150 and for a legitimate business purpose.

   Petty cash may not be used to avoid submitting purchase requests, check requests, or expense reports.

If the expense meets company policy, goto task #4. Otherwise, goto task #3.

3. Return the receipt to the Employee.

- Explain why the expense does not meet company policy.

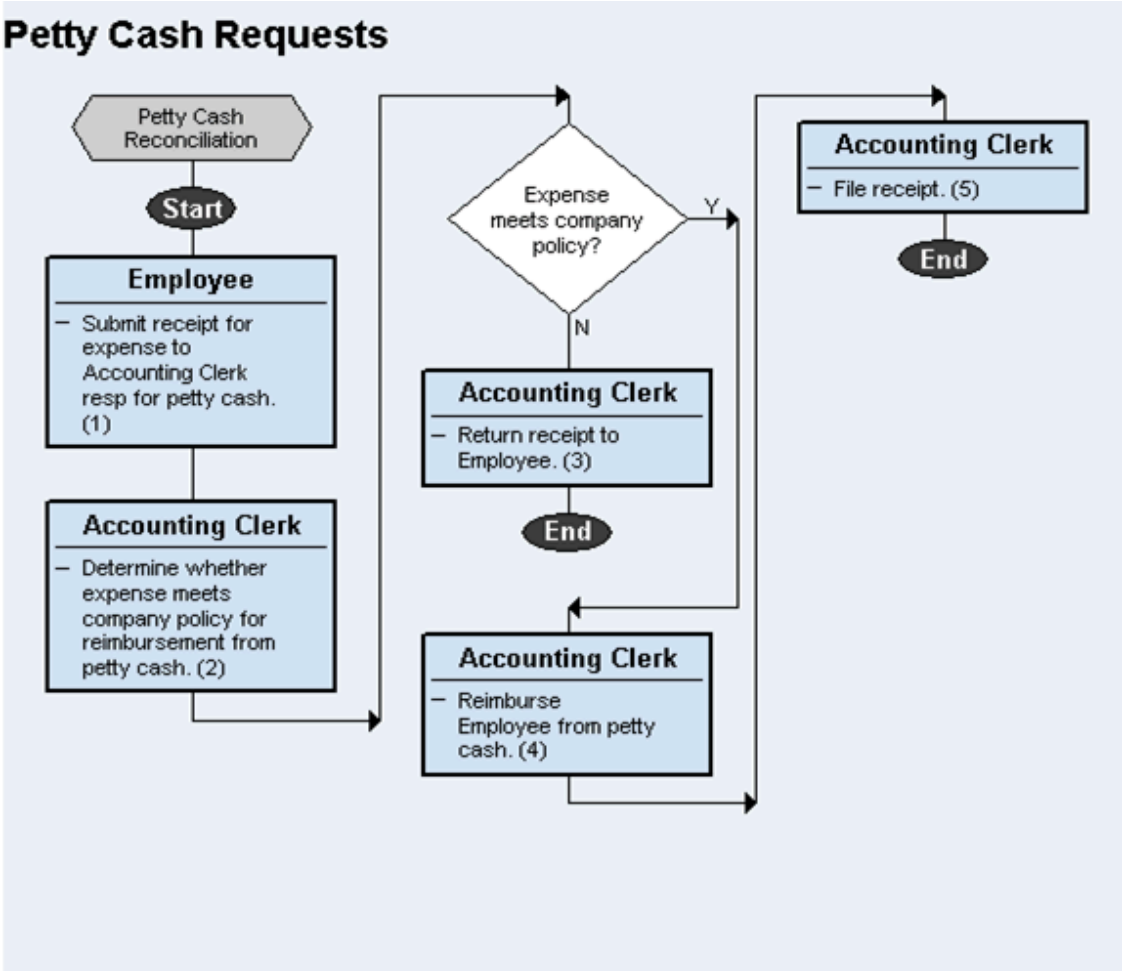
End of activity.

4. Reimburse the Employee from petty cash.

5. File the receipt.

End of activity.

Procedure Task List Section



Procedure Flowchart Section

Back to Top

DOCUMENTATION (DO) PROCESS GUIDANCE

DO.010 Define Documentation Requirements and Strategy

It is important to formally define the relationship of the documentation effort to the development of the future state model. During validation of the Functional Prototype (RA.085) in a project, the 'official' approach to using the Application in a specific environment is developed. This must be communicated effectively to the resources developing the procedures. The best approach is to have the documentation effort performed in concert with the validation of the Functional Prototype.

In a commercial off-the-shelf (COTS) application implementation, you could manage the documentation development after validation of the Functional Prototype. This is a less effective approach but in that case, it is crucial to identify in advance the means to communicate the future state model to the document developers, using detail system use cases, test scripts or an equivalent. Further, it is important to let the documentation team know in advance how this approach will be managed, to avoid attempts to execute a second Validate Functional Prototype activity.

Another key issue is whether the effort will include the Help Documents and Courseware as well as procedures and transaction recordings. The list of work products is:

- **Tutor** - Procedures and Business Process Flows. People to People work instructions.
- **UPK** - Help Document Modifications. For Oracle and non Oracle application projects, project teams can create custom online help with Oracle UPK or modify prebuilt UPK content. Publisher UPK output provides three learning methods where users can watch simulations, practice in a simulated environment, and test their knowledge to ensure system competency and increased productivity. Links to procedures and business process flows can be added to UPK content, making all levels of process information readily available to end users.
- **UPK** - Transaction recordings. People to System navigation instructions. If transaction recordings are developed with the User Productivity Kit, additional MS Word based documents are available automatically, as is the potential for using the output from UPK Developer exported into Mercury Quality Center, to create system process test scripts of the pre go live applications. It is wise to review the format and content of these documents, as they can be used for other work products without additional effort: Test Documents may be used as TE.040s, and System Process Documents from the prebuilt UPK content may be used during fit/gap analysis as well as end-user training materials.
- **UPK** - Courseware. Procedures provide instructions to users but stop at the screen level steps. Courseware is educational material that can be developed with UPK to instruct users how to use screens for their business. Courseware created with UPK can be used for a classroom environment, self paced training, and deployed as part of the Online Help for ongoing performance support, as a supplement to procedures.
- **Knowledge Center** - Portal based content deployment which may include Tutor documents, UPK recordings, other project documents, and student assessments based on the content deployed.

Oracle Tutor maintains baseline procedures known as Tutor Model Documents that are available to use as a starting point for procedure editing. Other baseline content is available that maps to Oracle Accelerators, or which is produced by outside parties. It is advisable to communicate which set or combination of documents has been chosen to all involved in the documentation effort, so that all parties will understand in advance which content will be the baseline. UPK content is also available for sale to customers. The scope of content is expanding, and the project team should research what is available at the time of the project.

Note that the scope of business procedures to be documented will be greater than the scope of Tutor Model Documents. Not all business processes requiring documentation reside in the business flows. If using Tutor Model Documents, use the Tutor Document Register (docreg.xls) found in the Tutor folder as a baseline for this.

DO.020 Define Documentation Standards and Procedures

While the Tutor Model Documents define a standard which is used as-is by many organizations, the content can be modified to suit the implementing organization. This applies to the prebuilt UPK transaction recordings and courseware documentation formats as well. Various issues are to be considered: Management must define how policies will be documented in the procedures. Document owners must define the degree of detail required to appropriately inform users. Auditors guide how the documents must be composed to meet compliance requirements (to eliminate duplicate efforts on the part of an audit team). Instructors must define courseware requirements working with users. Requirements must be gathered, defined, and agreed upon, then formatted into the document types.

One component of content standards in procedures is company policies. It can be useful to contact the HR or Finance departments within the implementing organization to review existing policies, and map them to business processes. In this way, policies such as purchasing dollar limits can be identified and prepared for inclusion in the proper procedures. The Tutor Report ‘Policy to Procedure’ (POL2PRO) is a useful tool for this activity.

It is recommended that if transaction recordings are being developed, the UPK Course Development Standards Guide be studied to assist the implementing organization in defining their templates for transaction recordings in their organization. An approach to linking Tutor Procedures to UPK Topics should be defined.

Key standards items to address include:

- Organization Role name standards
- Approach to integrating policies to procedures
- Approach to specifying department or division specific policies or instructions
- Document Owner responsibilities
- Procedure to Transaction Recording and Procedure to Applications Help linking
- Document storage, backup, and backup schedule

Note that standards may be continued to be refined through the early development activity, but should be settled prior to significant documentation effort.

DO.040 Prepare Documentation Environment

Install the Tutor software as appropriate on individual PCs and the development server. Author will be deployed on the PCs of the document developers. Publisher needs only to be loaded on a single environment. See the Tutor Implementation Guide found in the User Manuals directory of the Tutor Install.

Install the UPK Developer software as appropriate. Decisions made previously about central server or individual PC deployment will guide this activity. See the Installation and Administration Guide found in the UPK Developer 31/Documentation/EN/Reference directory.

Test document collection and backup mechanisms.

Note that formal document management is provided by such software made available by the implementing organization.

DO.070 Publish User Guide

The Tutor output that corresponds to the User Guide is the Desk Manual. The desk manual is automatically created by Tutor Publisher, and is the collected set of procedures and other process documents created by Tutor: navigation instructions, instructions, form abstracts, coding conventions, and reference documents. Publisher determines which process documents to include in a desk manual by analyzing each document's distribution list. The Stockroom Clerk's desk manual, for example, includes only those process documents that list Stockroom Clerk under Distribution. Creating desk manuals requires that all procedures be collected in a central working folder structure for access by Publisher.

There is also the option of creating a desk manual index. This document lists all of the job roles. A user can click on a specific job role to link directly to the desk manual for that role. A user who opens the online desk manual may jump from the table of contents to any of the documents that it references.

Desk manuals are created in a two step process - first an index of roles to documents is created by Publisher, then the desk manuals themselves are created. They are html documents that can be deployed on a web server for access by the document owners, the project team, and ultimately, the end users. Since creating the desk manuals takes little time, this can be done on a regular basis to monitor the progress of procedure development.

Examples of the User Guide work product are below. The first is the Desk Manual Index which contains all roles. By selecting a role, the user can navigate to an individual Desk Manual, which is the second example. From the Desk Manual, the user can access the individual procedures that apply to that role.

ORACLE Tutor

Desk Manuals | Process Flows | Document Register | Impact Analysis Reports

Desk Manual Index

Accountant
Accounting Clerk
Accounts Payable Clerk
Accounts Payable Supervisor
Accounts Receivable Clerk
Accounts Receivable Supervisor
Administrative Assistant
Assembler
Auditor
Benefits Analyst
Board of Directors
Budget Manager
Budget Specialist
Business Analyst
Business Development Manager
Buyer
Chairman of the Board
COBRA Plan Participant
Compensation Analyst
Compensation Manager
Component Engineer
Component Engineering Manager
Computer Operations Supervisor
Human Resources Manager
Human Resources Representative
Human Resources Specialist
Information Services Manager
Inventory Control Manager
Legal Affairs Representative
Manager
Manager's Manager
Manufacturing Clerk
Manufacturing Engineer
Manufacturing Engineering Manager
Marketing Support Clerk
Master Scheduler
Material Control Coordinator
Payroll Administrator
Payroll Clerk
President
Product Engineer
Product Manager
Product Marketing Manager
Production Control Manager
Production Control Planner
Project Accountant

Tutor Desk Manual Index

ORACLE Tutor

Desk Manuals | Process Flows | Document Register | Impact Analysis Reports

Desk Manual: Accounting Clerk

Quick link Procedures Go

Creating Manual Encumbrance Journal Entries
Employee Termination
General Ledger Period End Close - Adjustments
Maintaining Foreign Currency Rates
Monitoring the Close and Avoiding Delays

Journal Entry

Balance Sheet Account Reconciliation
Close Schedule
Collection Action Table

Procedures

Petty Cash Reconciliation
Petty Cash Requests
Processing Manual Journal Entries
Processing Subledger Data to the General Ledger
Relieving Manual Encumbrances

Business Forms

Reference Documents

General Ledger Account Number Table
Journal Entry Responsibility Table
Narrative Account Descriptions

The following steps guide the assembly and publication of the Desk Manuals.

No.	Task Step	Description
1	Publish Procedures.	See the following Tutor Documents for additional guidance: • INS1062Y, Building the Tutor Index • INS1064Y, Publishing Approved Tutor Documents • INS1080Y, Linking a Procedure to a Transaction Recording • INS1061Y, Capturing Application Help Targets See also the Tutor Publisher User Manual for specific How-To instructions.
2	Publish Desk Manuals.	See the Tutor Publisher User Manual for complete Instructions.
3	Publish UPK Transaction Recordings.	See the Tutor Publisher User Manual for complete Instructions.

DO.100 Produce Online Help

Oracle Applications contain Help documentation deployed as context sensitive html. When an end user is in a specific transaction and presses the help button, a help document opens which applies to that transaction. Oracle UPK can be used to create custom help that is also context sensitive for most web applications. In the UPK content, links can be added to procedures, making the business process documentation available to end users on a context sensitive basis as well. If the project includes the customization of Online Help as a work product, using UPK can be very useful. The custom created quickly from a single UPK authoring session can be used to generate multiple outputs that can be used for end-user training and testing, just to name a few.

No.	Task Step	Description
1	Conduct Applications Help Editing Workshop.	This workshop may be an extension of the procedure editing workshop.
2	Conduct UPK Developer Workshop.	During scope definition, an initial list of the transaction recording topics should have been developed and owners assigned. The list of topics may have been further refined during the procedure editing process, with priority going to the complex or critical transactions that support key procedures. In this workshop, additional review of topics and owners should be done with the UPK developer team. It may be appropriate to use the Document Register as a procedure/topic cross reference and tracking mechanism.
3	Develop UPK transaction recordings.	Transaction recordings are best developed once the functional prototype environment contains organization data, and the organization business processes have been defined to the point where the transaction recordings are depicting the valid future state model. This is typically the second iteration of functional prototyping or later. End users in training will more easily understand transaction recordings that contain data with which they are familiar.
4	Map transaction recordings to procedures.	During standards definition, the linking approach will have been defined. Typically a procedure will have a transaction contained in a task, and there will be a link to the appropriate transaction recording. Adding links to procedures and transaction recordings should occur after the deployment environments have been defined for both training and the Applications instance, so that the links are accurate. Review the Instruction INS1080Y, Linking a Procedure to a Transaction Recording, for information on appropriate setup of production environments to ensure accurate linking. Best practice is to deploy the transaction recordings into the production environment and use absolute links between from the procedures to the transaction recordings. In this way, procedures can be deployed into both a training environment and into the Applications

		instance without breaking the links. If links are required from transaction recordings to procedures, a similar scenario must be developed. For additional guidance, see the Tutor document, Link a Procedure to a UPK Help Topic [INS1080Y].
5	QA transaction recordings.	
6	QA Help Edits.	The QA cycle is performed until the content meets standards. For the verification of complete two-way referencing of procedures to Help documents, the QA must be coordinated with the Help Document QA process.
7	Publish transaction recordings and Word documents.	Transaction recordings are published as html-enabled recordings and deployed as html. The Word versions can be published simultaneously and used for a variety of purposes, including packaged in with the courseware. See the UPK document, Content Deployment.
8	Upload UPK Help to Applications Environment.	

Back to Top

ORGANIZATIONAL CHANGE MANAGEMENT (OCM) PROCESS GUIDANCE

OCM.140 Develop Communication Plan

The documentation project may take additional resources from the implementing organization user community. In addition, the training of end users needs to be communicated and scheduled. The Change Management Roadmap/Communication Campaign manages the communication of the organizational change management approach, requirements, and schedule to the manager and user communities.

OCM.180 Determine Impact of Job Changes

The work product for this task is the Job Impact Diagnosis. Tutor and UPK deliverables should be used to help define, by role within the organization, what is changing in terms of day-to-day work activity. Tutor procedures describe both the manual and systems supported tasks of the future state model. UPK transaction recordings identify the exact approach to processing transactions. Together with an assessment of what work activities will no longer be performed, a detailed guide to the impact of an implementation or a process improvement exercise can be communicated to managers.

OCM.210 Conduct Managers' Unit/Department Impact Workshop

The documentation project may take additional resources from the implementing organization user community. In addition, the training of end users needs to be communicated and scheduled. Present the documentation project strategy, resource requirements, schedule, and change management approach to business managers so they may prepare their organizations.

Point out that the most effective approach is to have the document owners come from within their organizations, in order to improve accuracy and uptake.

Back to Top

TRAINING (TR) PROCESS GUIDANCE

TR.020 Prepare Project Team Learning Plan

The documentation project team needs to be trained in using Tutor and UPK. Schedule Tutor and UPK Classes and Workshops. Confirm delivery of event kits.

TR.050 Conduct Project Team Learning Events

Project teams, consisting of both internal and external resources, need to learn key sets of knowledge: Oracle Applications (when appropriate), Oracle UPK Pro and Oracle Tutor. If there is Applications education scheduled for the project team, it should be scheduled before functional prototyping activities begin, so that the internal team is familiar with the software as the future state model is developed. If UPK pre built content is available to the project, this content is an excellent source for applications training, especially as it does not require an instance be set up yet simulates the application environment.

Project teams need to be educated with Tutor software as well. Project team members can conduct self-guided learning by reading the *Tutor Implementation Guide*, and familiarizing themselves with additional Tutor supplied reference material.

Based on the project team learning plan, Tutor classes or workshops from Oracle University or Oracle Consulting are presented to the project team to ensure that all team members have the Tutor skills necessary to develop content in the Construction phase.

This activity may segue directly into the Procedure Editing Workshop described in the Construction phase. In fact, it is preferable to conduct a joint class and procedure editing workshop where the documentation project team learns to use the software and initial edits or revisions are conducted under the eye of the instructor. In this fashion, everyone learns the correct approach together, and a more homogeneous set of procedures is developed. In this case, the Procedure Editing Workshop task in the Construction phase would be combined with this task.

UPK transaction recording development typically occurs during or after functional prototyping activities, once the training Applications instance contains test data of the implementing organization. Training for UPK can take place at any time after simulation standards are set and before functional prototyping activities.

If Knowledge Center is being used, the development of the guided learning portals can only occur once other content such as procedures and transaction recordings have been developed. Training for Knowledge Center can occur late in the project.

TR.080 Develop User Learningware

The work product, User Learningware corresponds to UPK assembled Courseware documents and UPK transaction recordings. There can be many components to user learningware:

- Baseline UPK Transaction Recordings - Oracle Application teams have developed baseline prebuilt content (transaction recordings) using UPK for many of the Oracle Applications concepts and products. This content must be licensed and can be downloaded from the Oracle Software Delivery Cloud. The prebuilt content contains source UPK player files, along with published Word documents. The source player content can be modified to meet the needs of the implementing organization. The prebuilt content should be used as is to train the project team, and modified once the future state model has been developed to eliminate superfluous material for the end user community. Once edited (or created from scratch), the player content is published and assembled into Instructor and Student Guides through Oracle UPK to be used in instructor led or self paced training.
- Tutor Procedures - These can be linked to the courseware or deployed as html. Development of procedures is covered above.

No.	Task Step	Description
1	Conduct Courseware Workshop.	Steps 2 - 6 may have been completed in the Help Documentation Workshop if UPK was used. If UPK was used for Help Documentation, the time and effort to develop learningware will be greatly reduced.

2	Conduct UPK Developer Workshop to develop courseware.	During scope definition, an initial list of the transaction recording topics should have been developed and owners assigned. The list of topics may have been further refined during the procedure editing process, with priority going to the complex or critical transactions that support key procedures. In this workshop, additional review of topics and owners should be done with the UPK developer team. It may be appropriate to use the Document Register as a procedure/topic cross reference and tracking mechanism.
3	Develop UPK transaction recordings.	Transaction recordings are best developed once the functional prototype environment contains organization data, and the organization business processes have been defined to the point where the transaction recordings are depicting the valid future state model. This is typically the second iteration of functional prototyping or later. End users in training will more easily understand transaction recordings that contain data with which they are familiar.
4	Map transaction recordings to procedures.	During standards definition, the linking approach will have been defined. Typically a procedure will have a transaction contained in a task, and there will be a link to the appropriate transaction recording. Adding links to procedures and transaction recordings should occur after the deployment environments have been defined for both training and the Applications instance, so that the links are accurate. Review the Instruction INS1080Y, Linking a Procedure to a Transaction Recording, for information on appropriate setup of production environments to ensure accurate linking. Best practice is to deploy the transaction recordings into the production environment and use absolute links between from the procedures to the transaction recordings. In this way, procedures can be deployed into both a training environment and into the Applications instance without breaking the links. If links are required from transaction recordings to procedures, a similar scenario must be developed. For additional guidance, see the Tutor document, Link a Procedure to a UPK Help Topic [INS1080Y].
5	QA transaction recordings.	
6	Publish Transaction Recordings and Word Documents.	Transaction Recordings are published as html-enabled recordings and deployed as html. The Word versions can be published simultaneously and used for a variety of purposes, including packaged in with the courseware. See the UPK document, Content Deployment.
7	Publish Courseware.	See the UPK Content Development: • Publish Content > Document Formats > Training Guide • Publish Content > Document Formats > Instructor Manual

---

TR.090 Prepare User Learning Environment

It may be appropriate to prepare different environments for the user community as people learn in different modes:

- A server based Desk Manual Repository without access to an Oracle Applications instance can be useful for regular deployment of Work in Process content for review by key members of the user community.
- An Oracle Applications Training instance with the procedures and modified Help documents loaded will enable users to use the procedures and courseware to generate training transactions without impacting a production instance.
- A classroom environment with access to either or both of the above to facilitate Instructor led training.
- Deployment of content via Learning Management systems will help track employee readiness, especially if assessments are part of the overall curriculum.

If Knowledge Center is used as a content deployment tool, the following steps must be taken to roll out the content:

No.	Task Step	Description
1	Conduct Knowledge Center Workshop.	Limit this to the In-House Knowledge Center course designer(s).
2	Develop Workgroups.	Typically, only the master workgroup is required, others may be useful if there is a completely independent business unit within an organization.
3	Load content.	
4	Define knowledge paths.	
5	Define Usergroups.	Group users by role or track.
6	Assign and create assessments.	Define success criteria with project team and organizational management.
7	QA Transaction Recordings.	
8	Create custom reports.	
9	Finalize content deployment environment.	

TR.100 Conduct User Learning Events

Content developed with Oracle Tutor and UPK, and deployed via UPK Pro Knowledge Center, is suited for both self service, on demand learning, and instructor led learning, Further, the ability to link Tutor Process Models and Procedures, and UPK transaction recordings into the Oracle Applications, make these work products available for production support after go-live.

Back to Top

OPERATIONS AND SUPPORT (PS) PROCESS GUIDANCE

PS.010 Audit System

Post Go-Live system auditing is not a one time event. The implementing organization should be instructed to audit the system, procedures, and training material to ensure accurate reflection of changing business conditions. Proactive auditing helps make certain that the Applications are being used correctly and efficiently and supports productivity.

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.